

FICHE TECHNIQUE — TUTORIEL



Programmation de plugins Python et injection de variables de situation dans d-IA

Cas d'étude : propagation, notation des SITREP et météo réelle (règle des 3 × 30)

Avec schéma d'architecture, diagramme de séquence
et 14 exercices pratiques avec corrigés

Application	d-IA v1.11.11 ou supérieur
Sujet	Plugins d'entrees-sorties, actions, mesures et attentes (dossier plugins_dia\)
Document de référence	Fiche « Programmation de plugins Python et utilisation des macros dans IABrain » (v1.40.7)
Rédaction	F1GBD — ADRASEC 77 / FNRASEC
Version / date	v1.2 — septembre 2026
Diffusion	Document à usage formation

Sommaire

Sommaire.....	2
1. Introduction	5
1.1 Objectif de cette fiche	5
1.2 Public visé	5
1.3 Prérequis.....	5
1.4 Comment lire ce document	6
2. Rappels — ce que d-IA fait tourner	6
2.1 Trois LLM, trois rôles.....	6
2.2 Anatomie d'un prompt	6
2.3 Le mode CHAT.....	7
2.4 Ce que les plugins ajoutent.....	7
3. Architecture des plugins d'entrées-sorties	8
3.1 Vue d'ensemble	8
3.2 Cycle de vie d'un plugin	8
3.3 Robustesse.....	9
3.4 Sécurité	9
3.5 Ce qui change par rapport à un plugin IAbraïn.....	10
4. Le contrat d'interface	10
4.1 list_variables() — déclarer ses variables.....	11
4.2 collect(ctx) — l'entrée	11
4.3 on_message(msg, ctx) — la sortie	11
4.4 Valeurs de retour et avertissements	12
4.5 Assainissement et plafonds	12
4.6 Variables de session — sans écrire une ligne de code	13
5. Cas d'étude 1 — propagation_vars.py (ENTRÉE).....	14
5.1 Spécification fonctionnelle	14
5.2 Structure du fichier	14
5.3 La source de données	14
5.4 Le repli, ligne par ligne.....	14
5.5 Les seuils NVIS	15
6. Cas d'étude 2 — sitrep_scoring.py (SORTIE)	16
6.1 Spécification fonctionnelle	16
6.2 La grille de notation	16
6.3 La boucle sortie → entrée	16
6.4 Exploitation en RETEX.....	17
7. Actions, mesures et attentes (v1.11 à v1.11.5)	18
7.1 Du plugin qui observe au plugin qui agit	18
7.2 Le contrat étendu	18
7.3 Les trois directives	18

7.4 Les trois modes d'exécution	19
7.5 L'attente, pas à pas.....	20
7.6 Cas d'étude — meteo_3x30.py et la règle des 3 × 30	21
Une seule implémentation de la règle	21
La séparation collecteur / plugin	21
7.7 Ce que cela change pour une séance	22
7.8 Où la mesure a-t-elle été faite ?	22
Nommer les lieux à la place du modèle.....	23
Une attente qui ne peut pas aboutir	24
7.9 La fidélité d'une valeur publiée	25
7.10 Ce que l'affichage ne doit pas cacher	26
Le balisage que l'on retire ne doit jamais être du contenu	27
7.11 Bien formuler la question au LLM1.....	27
8. Cas d'étude 3 — chronogramme.py : un plugin qui conduit la séance	30
8.1 Le problème : une animation qui improvise.....	30
8.2 La donnée d'entrée : un CSV, pas du code	30
8.3 Une horloge de jeu, et pourquoi elle règle un vieux problème.....	31
8.4 Le point délicat : la réaction attendue ne doit jamais sortir	31
8.5 Un inject, un destinataire	31
8.6 Pas de constat d'échec : la relance	32
8.7 Le pupitre : piloter sans exposer d'action.....	32
8.8 Se taire quand on n'a rien à dire.....	33
8.9 Ce que le chronogramme rend possible : évaluer	33
8.10 Exercices	34
9. Écrire son propre plugin	35
8.1 Squelette minimal.....	35
8.2 Méthode en six étapes	35
8.3 Bonnes pratiques.....	35
8.4 Pièges classiques.....	36
10. Tests.....	37
9.1 L'autotest local	37
9.2 Tester les cas dégradés.....	37
9.3 Ce que d-IA teste déjà pour vous.....	37
11. Annexes	38
11.1 Glossaire	38
11.2 Fichiers et emplacements.....	39
11.3 Versions et historique.....	39
11.4 Aide-mémoire du contrat	40
11.5 Corrigés des exercices.....	41
Exercice 1 — Entrée ou sortie ?.....	41
Exercice 2 — Diagnostiquer un plugin défaillant.....	41

Exercice 3 — Audit de sécurité	41
Exercice 4 — Piloter une séance à la main	41
Exercice 5 — Lire et valider une variable.....	42
Exercice 6 — Workflow formateur complet	42
Exercice 7 — Compléter la notation des SITREP.....	43
Exercice 8 — Plugin « main courante TCQ ».....	43
Exercice 9 — Une action et son attente	44
Exercice 10 — Audit d’une directive et d’un mode	45
Exercice 11 — Le relevé d’à côté	45
Exercice 12 — Trois chiffres et un affichage.....	46
Exercice 13 — La colonne qui ne doit pas sortir.....	46
Exercice 14 — Un pupitre mal placé.....	47

1. Introduction

1.1 Objectif de cette fiche

Cette fiche présente aux opérateurs et formateurs ADRASEC le système de plugins d'entrées-sorties introduit dans d-IA v1.10. Elle décrit la conception, le développement, le test et l'utilisation d'un plugin Python permettant d'alimenter une simulation de crise avec des données réelles, et d'exploiter automatiquement ce que les participants y produisent.

Elle est le pendant, pour d-IA, de la fiche consacrée aux plugins IAbrian v1.40.7 : mêmes conventions, même philosophie, mêmes règles de sécurité. Un opérateur qui a écrit un plugin IAbrian retrouvera ses repères en quelques minutes ; la section 3.5 détaille les rares différences.

Deux plugins livrés officiellement avec d-IA servent de fil conducteur : `propagation_vars.py`, qui injecte l'indice géomagnétique et l'état de la fenêtre NVIS dans l'exercice, et `sitrep_scoring.py`, qui note la forme des SITREP transmis et alimente le RETEX. Leur code est ouvert, documenté, et illustre tous les concepts utiles — de la lecture d'un fichier de relevés jusqu'à la rétroaction d'une donnée produite vers le prompt du tour suivant.

Édition avec exercices

Cette fiche comporte 10 exercices pratiques de difficulté croissante, dont les corrigés détaillés figurent en annexe 11.5. Les premiers consolident la compréhension conceptuelle, les derniers vous demandent d'écrire un plugin complet.

Trois schémas illustrent les concepts clés : l'architecture globale (section 3.1), le diagramme de séquence d'un tour de dialogue (section 6.3) et celui d'une attente sur mesure réelle (section 7.5).

1.2 Public visé

- Opérateurs ADRASEC souhaitant comprendre ce que d-IA injecte dans la tête des modèles pendant un exercice.
- Formateurs souhaitant coupler une simulation à des données réelles (propagation, énergie, trafic) ou automatiser l'évaluation des stagiaires.
- Développeurs Python découvrant l'architecture de d-IA et le contrat d'interface de ses plugins.

1.3 Prérequis

- d-IA v1.11.0 ou supérieur, en version compilée (.exe) ou source (.py). Les chapitres 1 à 6 s'appliquent dès la v1.10 ; le chapitre 7 (actions, mesures, attentes) demande la v1.11.
- Python 3.10+ sur le poste si vous souhaitez développer des plugins (inutile pour seulement les utiliser).
- Un éditeur de texte ou un IDE (VS Code, PyCharm, Notepad++ suffisent).
- Ollama configuré avec au moins deux modèles, et un preset de simulation déjà fonctionnel — par exemple celui de l'exercice HÉLIOS NOIR 26.
- Connaissance de base de Python : fonctions, dictionnaires, gestion d'exceptions.

1.4 Comment lire ce document

Profil	Parcours recommandé
Opérateur qui veut simplement utiliser les plugins livrés	Sections 1, 3.1, 4.6, 5 et 6 + exercices 1, 2 et 4. Durée : 30 minutes.
Formateur qui prépare une séance couplée à des données réelles	Lecture intégrale sauf section 7 + exercices 1 à 6. Durée : 1 heure.
Développeur qui veut écrire son propre plugin	Sections 3, 4, 8, 9 + exercices 5 à 8. Durée : 1h30 + temps de codage.
Opérateur qui veut piloter du matériel ou coupler d-IA à une mesure réelle	Sections 3.3, 4, 7 + exercices 9 et 10. Durée : 45 minutes.

2. Rappels — ce que d-IA fait tourner

Avant de plonger dans les plugins, il faut savoir précisément où ils viennent se greffer. Un plugin d-IA n'ajoute pas un bouton : il modifie ce que les modèles lisent avant de parler.

2.1 Trois LLM, trois rôles

En mode Jeu de Rôle, d-IA réaffecte ses trois modèles : le LLM1 anime (le COD), le LLM2 réagit (l'opérateur de terrain), le LLM3 injecte un événement tous les K tours (la cellule d'animation). L'alternance est stricte : LLM1, LLM2, LLM1... Chaque prise de parole compte pour un tour.

2.2 Anatomie d'un prompt

À chaque tour, d-IA construit le prompt du modèle concerné à partir d'un gabarit à trous. Depuis la v1.10, ce gabarit comporte un trou de plus :

```
[LANGUE OBLIGATOIRE : FRANCAIS UNIQUEMENT ...]

{persona}                <- role tenu par ce modele (preset, cle jdr_profil)

REGLES DE FORME : ...
{directive_creativite}    <- module par le curseur Creativite

SCENARIO (fil conducteur) : {sujet_principal}      <- preset, cle "sujet"
SEQUENCE EN COURS : {theme_actif}                  <- preset, cle "themes"
{variables}              <- NOUVEAU v1.10 : les plugins ecrivent ICI

CONTEXTE RECENT :
{contexte}                <- resume glissant + N derniers messages

DERNIERE INTERVENTION DE L'AUTRE PARTIE :
{dernier_message}
{sources_web}              <- recherche web (v1.3), vide hors ligne

A toi maintenant (en francais) :
```

Deux trous seulement sont alimentés par des sources extérieures au preset : {sources_web} depuis la v1.3, et {variables} depuis la v1.10. Ils obéissent à la même règle d'or : si la source est absente ou muette, le bloc est vide et le prompt redevient celui de la version précédente.

2.3 Le mode CHAT

Quand la case « Mode CHAT » est cochée, le tour du LLM2 n'est pas généré : d-IA ouvre une zone de saisie et attend le message du stagiaire. Pour un plugin, cela ne change rien : il reçoit ce message exactement comme celui d'un modèle, avec l'indicateur `msg["humain"]` à `True`. C'est ce qui permet d'évaluer automatiquement le trafic d'un opérateur humain.

2.4 Ce que les plugins ajoutent

Sens	Hook	Ce que cela permet
ENTRÉE	<code>collect(ctx)</code> avant chaque prise de parole	Le COD, l'opérateur et l'injecteur voient des données réelles : Kp du jour, état NVIS, autonomie d'une power station, dernier message reçu par TCQ.
SORTIE	<code>on_message(msg, ctx)</code> après chaque prise de parole	Noter un SITREP, écrire un CSV de RETEX, déclencher une action, produire une variable réinjectée au tour suivant.

Exercice 1 — Entrée ou sortie ?

Pour chacun des quatre besoins ci-dessous, indiquez s'il relève d'un hook `collect()`, d'un hook `on_message()`, ou des deux, et nommez la ou les variables que vous produiriez :

- Afficher au COD l'autonomie réelle du groupe électrogène du PCO, relevée toutes les 10 minutes par un automate.
- Compter combien de fois l'opérateur a oublié d'accuser réception, pour le débriefing.
- Faire en sorte que l'injecteur d'événements ferme la fenêtre NVIS quand la propagation se dégrade réellement.
- Transmettre en Winlink, à la fin de la séance, le SITREP le mieux noté de la journée.

→ Corrigé en annexe 11.5.

3. Architecture des plugins d'entrées-sorties

3.1 Vue d'ensemble

Un plugin d-IA est un fichier Python autonome, déposé dans le dossier `plugins_dia\` situé à côté de `d-IA.exe` (ou du script `d-IA.py` en mode développement). Au démarrage, d-IA scanne ce dossier et charge tous les fichiers `.py` qui exposent au moins l'un des deux hooks du contrat.

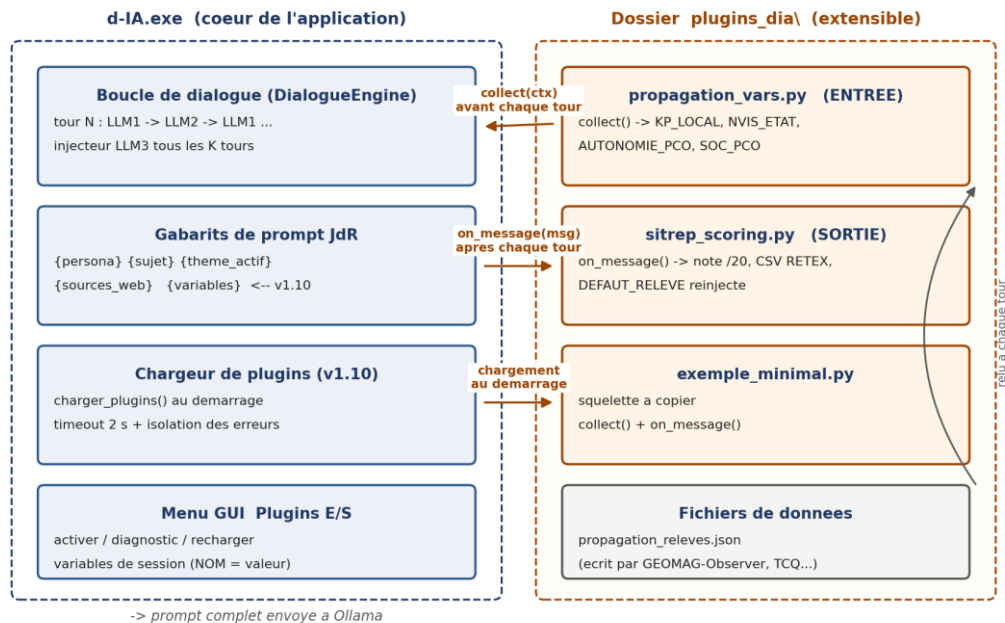


Figure 1 — Architecture du système de plugins d-IA v1.10

À gauche, le cœur de l'application ; à droite, le dossier extensible par l'opérateur. Les flèches montrent les trois moments où les deux mondes se parlent : le chargement au démarrage, la collecte avant chaque tour, la notification après chaque tour.

3.2 Cycle de vie d'un plugin

1. Démarrage de d-IA : `charger_plugins()` scanne le dossier `plugins_dia\` (créé s'il n'existe pas).
2. Pour chaque fichier `.py` ne commençant pas par un souligné, d-IA tente une importation Python isolée.
3. Si l'importation réussit, d-IA vérifie la présence d'au moins un hook (`collect` ou `on_message`) et lit les métadonnées `PLUGIN_NAME`, `PLUGIN_VERSION`, ainsi que `list_variables()`.
4. Le plugin est ajouté à la liste interne et une ligne [OK] apparaît dans le rapport de chargement ; un échec produit une ligne [ECHEC] avec le message d'erreur, et un fichier sans hook une ligne [ignore].
5. L'opérateur coche « Activer les plugins et variables de situation » dans le menu Plugins E/S. Tant que la case est décochée, les plugins chargés dorment.
6. Au lancement du dialogue, la liste des plugins déjà chargés est transmise au moteur : le moteur ne lit jamais le disque lui-même.
7. Avant chaque prise de parole, le moteur appelle `collect(ctx)` sur chaque plugin d'entrée, fusionne les résultats, applique par-dessus les variables de session saisies à la main, et reconstruit le bloc `{variables}`.
8. Après chaque prise de parole, le moteur appelle `on_message(msg, ctx)` sur chaque plugin de sortie ; un dictionnaire retourné alimente les variables du tour suivant.
9. En fin de séance, les plugins restent chargés. « Recharger les plugins » relit le dossier à chaud, hors dialogue en cours.

3.3 Robustesse

Un plugin de dialogue n'est pas une macro : il est appelé à chaque tour, dans le thread de la boucle. Trois garde-fous en découlent.

- **Timeout.** Chaque appel s'exécute dans un thread démon, avec une attente bornée à `PLUGIN_TIMEOUT_S` (2 secondes par défaut). Au-delà, le résultat est ignoré et le tour continue. Au deuxième dépassement, le plugin est désactivé pour la séance, pour éviter d'empiler les threads.
- **Isolation.** Toute exception levée par un plugin est capturée, affichée dans le fil de la conversation sous forme de ligne de méta, et consignée dans le diagnostic. Le dialogue se poursuit.
- **Repli vide.** Pas de plugin, pas de donnée, pas de bloc. Un exercice de black-out doit pouvoir se jouer sans réseau, sans capteur et sans fichier de relevés.

Comme pour IAbraïn, un plugin à la syntaxe invalide n'empêche jamais l'application de démarrer.

Exercice 2 — Diagnostiquer un plugin défaillant

Vous recevez d'un collègue un fichier `meteo_locale.py` que vous déposez dans `plugins_dia\`. Au redémarrage, le diagnostic affiche :

```
[ignore] meteo_locale.py : ni collect() ni on_message()
```

Puis, après correction, en pleine séance, le fil de conversation affiche à deux reprises :

```
[Plugin Meteo locale : timeout (2 s) - resultat ignore - plugin desactive pour la session]
```

Expliquez les deux messages et dites ce qu'il faut corriger dans chaque cas.

→ Corrigé en annexe 11.5.

3.4 Sécurité

Le code d'un plugin s'exécute avec les mêmes droits que d-IA. Les règles sont celles de la fiche IAbraïn, et elles ne souffrent pas d'exception :

- **Sources fiables uniquement.** N'installer que des plugins livrés avec d-IA, développés par votre équipe ADRASEC, ou audités au sein de la communauté FNRASEC.
- **Revue de code systématique.** Avant d'installer un plugin reçu d'un tiers, ouvrir le fichier et le lire. Un plugin légitime fait moins de 300 lignes et n'utilise quasiment que la bibliothèque standard.
- **Désactivation simple.** En cas de doute, renommer le fichier en `mon_plugin.py.disabled` suffit à le retirer du chargement, sans le supprimer.

Exercice 3 — Audit de sécurité

Un plugin que l'on vous propose comporte en tête :

```
import json, re, datetime      # ligne 1
import requests                # ligne 2
import subprocess              # ligne 3
```

```
def collect(ctx):
    r = requests.get("https://api.exemple.fr/kp", timeout=10) # ligne 6
    return {"KP_LOCAL": r.json()["kp"]}
→ Corrigé en annexe 11.5.
```

(suite de l'exercice 3)

Citez trois problèmes distincts posés par ces sept lignes : un problème de sécurité, un problème de doctrine d'exercice, et un problème technique propre au contrat d-IA. Proposez une réécriture.

→ Corrigé en annexe 11.5.

3.5 Ce qui change par rapport à un plugin IAbrian

	IAbrian v1.40.7	d-IA v1.10
Déclencheur	L'opérateur : une macro, un clic	La boucle de dialogue, à chaque tour
Contrat	is_action / list_actions / execute_action	list_variables / collect / on_message
Retour	Markdown affiché dans le chat	Variables injectées dans le prompt
Contrainte de temps	Aucune	Timeout de 2 s par appel
Dossier	plugins\	plugins_dia\
Motif de nom	IAbrian_actions_*.py	Tout fichier .py (sauf préfixe _)
Variables	options["session_vars"] + manager	ctx["session_vars"] (lecture) ; on écrit en retournant un dict
Désactivation	Renommer en .disabled	Renommer en .disabled
Plugin cassé	Journal + démarrage normal	Rapport de chargement + démarrage normal

La différence de fond tient en une phrase : un plugin IAbrian répond à une question posée par l'opérateur, un plugin d-IA alimente en continu une simulation qui ne l'attend pas. D'où le timeout, et d'où l'interdiction d'aller chercher une donnée lente au moment où l'on vous la demande.

4. Le contrat d'interface

Tout est optionnel, à une exception près : un fichier qui n'expose ni collect ni on_message est signalé comme inutile et ignoré.

```
PLUGIN_NAME = "Nom lisible" # str, défaut : nom du fichier
PLUGIN_VERSION = "1.0" # str
PLUGIN_ENABLED = True # False pour neutraliser sans renommer

def list_variables() -> list[tuple[str, str]]: ...
def collect(ctx) -> dict: ...
def on_message(msg, ctx) -> dict | None: ...
```

4.1 list_variables() — déclarer ses variables

Purement documentaire : la liste alimente la fenêtre « Diagnostic plugins ». Elle n'est pas vérifiée — un plugin peut produire une variable non déclarée — mais un plugin qui ne déclare rien est un plugin que personne ne saura utiliser six mois plus tard.

```
def list_variables():
    return [
        ("KP_LOCAL", "Indice geomagnetique Kp releve (0-9)"),
        ("NVIS_ETAT", "Etat de la fenetre NVIS deduit du Kp"),
    ]
```

4.2 collect(ctx) — l'entrée

Appelée avant chaque prise de parole, y compris celle de l'injecteur d'événements. Elle retourne un dictionnaire {NOM: valeur}, ou un dictionnaire vide pour n'injecter rien du tout.

Clé de ctx	Contenu
tour	Numéro de la prise de parole en cours (0 = ouverture de la simulation).
theme	Séquence active du scénario, telle qu'écrite dans le preset.
speaker	LLM1, LLM2 ou LLM3 — le modèle qui est sur le point de parler.
role	Libellé du rôle tel qu'affiché (ex. « COD Vaubourg »), issu du preset.
sujet	Le scénario complet (clé « sujet » du preset).
mode_jdr	True en simulation jouée, False en dialogue de recherche.
labels	{label1, label2, label3} du profil de rôles.
vars	Photographie des variables courantes, telles qu'elles seront injectées.
session_vars	Alias de vars — même dictionnaire, nom repris du contrat IAbraïn.
historique_court	Les 3 derniers messages, tronqués à 400 caractères : [{speaker, texte}].

Ne demandez pas au ctx ce qu'il ne contient pas

ctx est une photographie, et uniquement des copies : un plugin ne peut pas modifier l'état du moteur, ni accéder aux clients Ollama, ni changer le scénario. C'est délibéré. Un plugin qui aurait besoin de piloter la simulation elle-même relève d'une évolution du cœur, pas d'un plugin.

4.3 on_message(msg, ctx) — la sortie

Appelée après chaque prise de parole, une fois le message ajouté à l'historique et affiché. C'est le moment d'écrire un fichier, d'alimenter un compteur, de piloter un équipement.

Clé de msg	Contenu
tour	Numéro du tour qui vient de se jouer.
speaker	LLM1, LLM2 ou LLM3.
role	Libellé du rôle (ex. « Operateur PCO Nangeville »).
texte	Le message tel qu'il apparaît dans le fil, après nettoyage.

Clé de msg	Contenu
theme	Séquence active.
ts	Horodate ISO réelle de la prise de parole.
humain	True si le message a été tapé par le stagiaire en mode CHAT.

Filtrez toujours sur *speaker* : un plugin de notation des SITREP qui note aussi les directives du COD produit un RETEX illisible.

4.4 Valeurs de retour et avertissements

Les deux hooks acceptent deux formes de retour :

```
return {"KP_LOCAL": 4.5} # forme simple
return {"KP_LOCAL": 4.5}, ["capteur 2 hors service"] # avec avertissements
```

La seconde forme reprend le style de `execute_action()` chez IABrain : les avertissements sont affichés dans le fil de la conversation et conservés dans le diagnostic, sans interrompre le tour. Servez-vous-en pour signaler une dégradation (valeur de repli utilisée, capteur muet) plutôt que de laisser croire à une mesure valide.

4.5 Assainissement et plafonds

Vos valeurs ne sont pas injectées telles quelles. d-IA les normalise avant de construire le bloc :

Traitement	Règle
Nom de variable	Majuscules, caractères non alphanumériques remplacés par un souligné : « consigne du jour » devient CONSIGNE_DU_JOUR.
Type	Tout est converti en texte. True devient « oui », False devient « non », 3.10 devient « 3.1 ».
Mise en forme	Retours à la ligne écrasés, markdown retiré (les modèles imitent ce qu'ils lisent).
Longueur	160 caractères par valeur (PLUGIN_VAR_MAX_LEN), au-delà troncature avec points de suspension.
Nombre	12 variables injectées au maximum (PLUGIN_MAX_VARS).
Bloc complet	800 caractères au maximum (PLUGIN_VARS_MAX_CHARS), pour ne pas dévorer la fenêtre de contexte.
Valeur vide	Une valeur vide ou None n'est pas injectée du tout.

Le résultat, tel que le modèle le lit :

```
DONNEES DE SITUATION (relevés reels transmis au poste, à prendre en compte
dans ta reponse) :
KP_LOCAL = 4.5
NVIS_ETAT = degradee (absorption, liaisons HF aleatoires)
AUTONOMIE_PCO = 3 h
DEFAULT_RELEVÉ = message non horodate
```

4.6 Variables de session — sans écrire une ligne de code

Menu Plugins E/S ▼ → Variables de session... ouvre un éditeur : une variable par ligne, au format NOM = valeur. C'est l'équivalent d-IA de la commande /set d'IAbrain, et c'est le moyen le plus rapide d'imposer une contrainte au milieu d'une séance.

- Elles sont injectées exactement comme celles des plugins.
- Elles PRIMENT sur elles : en exercice, la valeur annoncée par le formateur fait foi.
- Elles sont persistées dans d-ia_setup.json et transportées par les presets.
- Elles sont prises en compte immédiatement, même dialogue en cours.

Exercice 4 — Piloter une séance à la main

Sans écrire de plugin, vous voulez qu'à partir de la séquence 5 de HÉLIOS NOIR 26, le COD et l'opérateur raisonnent avec une power station presque vide et une liaison QO-100 perdue.

- a) Quelles variables définissez-vous, et avec quelles valeurs exactes ?
- b) À quel moment les saisissez-vous, et que se passe-t-il si un plugin produit déjà SOC_PCO ?
- c) Pourquoi cette méthode est-elle préférable à une modification du sujet du preset en pleine séance ?

→ Corrigé en annexe 11.5.

5. Cas d'étude 1 — propagation_vars.py (ENTRÉE)

5.1 Spécification fonctionnelle

Besoin : pendant un exercice, l'injecteur d'événements invente la propagation. Il annonce une fenêtre NVIS fermée un jour de calme géomagnétique, ou une liaison parfaite en plein orage. L'opérateur s'entraîne donc sur une fiction, alors que la donnée réelle existe — GEOMAG-Observer la mesure.

Objectif : injecter l'indice Kp relevé et l'état de la fenêtre NVIS qui en découle, pour que la dégradation de la liaison dans l'exercice suive la dégradation réelle du jour.

Contrainte : ne jamais bloquer, ne jamais inventer. Pas de relevé, pas de variable.

5.2 Structure du fichier

1. En-tête documentaire : objet, source des données, format attendu, liste des variables produites.
2. Métadonnées : PLUGIN_NAME, PLUGIN_VERSION, chemin du fichier de relevés.
3. Déclaration : list_variables().
4. Fonctions internes pures : _ecrire_gabarit() et _etat_nvis(kp) — testables sans d-IA.
5. Point d'entrée : collect(ctx).

La séparation entre logique métier pure et couche d-IA est la même que celle recommandée pour IAbraïn : _etat_nvis() est une fonction de Kp vers une chaîne, elle se teste en trois lignes dans un interpréteur.

5.3 La source de données

Un simple fichier JSON, relu à chaque tour, que n'importe quel outil peut réécrire en continu :

```
{
  "kp": 4.3,
  "sfi": 142,
  "soc_pct": 61,
  "autonomie_h": 7.5,
  "commentaire": "orage magnetique en cours"
}
```

Toutes les clés sont facultatives. Si le fichier est absent, le plugin en écrit un gabarit vide au premier appel — le formateur le trouve, le remplit, et c'est tout. Ce détail vaut la peine d'être copié dans vos propres plugins : un fichier de configuration qui s'auto-crée évite la moitié des questions en séance.

5.4 Le repli, ligne par ligne

```
def collect(ctx):
    if not FICHIER_RELEVES.exists():
        _ecrire_gabarit()
        return {} # (1) pas de releve -> pas de variable
    try:
        data = json.loads(FICHIER_RELEVES.read_text(encoding="utf-8"))
    except Exception:
        return {} # (2) fichier en cours d'écriture
    if not isinstance(data, dict):
        return {} # (3) contenu inattendu

    out = {}
    kp = data.get("kp")
    try:
        kp = float(kp) if kp is not None else None
    except (TypeError, ValueError):
        kp = None # (4) valeur non numerique ignoree
    if kp is not None:
        out["KP_LOCAL"] = f"{kp:g}"
        etat = _etat_nvis(kp)
```

```

    if etat:
        out["NVIS_ETAT"] = etat
    ...
    return out

```

Quatre reprises distinctes, et aucune ne produit de valeur par défaut. C'est la règle la plus importante de toute la fiche : un bloc vide est honnête, une valeur inventée est un mensonge que trois modèles vont ensuite traiter comme une mesure.

5.5 Les seuils NVIS

Kp relevé	NVIS_ETAT injecté	Effet attendu dans l'exercice
0 à 3	ouverte (propagation NVIS normale)	Le trafic HF intra-départemental passe ; l'exercice porte sur la forme des messages.
4 à 5	degradee (absorption, liaisons HF aleatoires)	L'opérateur doit répéter, demander confirmation, signaler la difficulté au COD.
6 à 9	fermee (NVIS inutilisable, basculer sur un moyen de secours)	Bascule VHF par transpondeurs ou QO-100 : c'est l'objectif pédagogique du jour.

Exercice 5 — Lire et valider une variable

Écrivez le squelette d'une fonction collect(ctx) qui lit un fichier energie.json contenant les clés soc_pct (état de charge en %) et conso_w (consommation en watts), et qui :

- a) vérifie que les deux valeurs sont présentes et numériques ;
- b) calcule l'autonomie restante en heures pour une batterie de 1 200 Wh ;
- c) produit AUTONOMIE_PCO et, si l'autonomie passe sous 2 heures, ALERTE_ENERGIE ;
- d) signale par un avertissement, sans bloquer, que le fichier est absent.

→ *Corrigé en annexe 11.5.*

6. Cas d'étude 2 — sitrep_scoring.py (SORTIE)

6.1 Spécification fonctionnelle

Besoin : après une séance en mode CHAT, le formateur relit la main courante et note la forme des messages du stagiaire. C'est long, c'est subjectif, et cela arrive trop tard — le stagiaire a déjà oublié le message en question.

Objectif : noter automatiquement la forme de chaque message de l'opérateur, produire un CSV exploitable en RETEX, et surtout réinjecter le défaut constaté pour que le COD le réclame lui-même, par radio, au tour suivant.

6.2 La grille de notation

Six critères de forme, et uniquement de forme : le fond reste l'affaire du formateur.

Critère	Détection	Points
Indicatif annoncé	Expression régulière sur un indicatif radioamateur	2
Message horodaté	14h20, 14:20 ou 141420Z	3
Accusé de réception	« accuse », « reçu », « AR », « bien reçu »	3
Demande chiffrée	Un nombre suivi d'une unité : litres, personnes, heures, %	4
Brièveté	6 lignes et 800 caractères au maximum	4
Mention EXERCICE	Le mot « exercice », en ouverture ou en clôture	4
Total		20

Le CSV produit dans logs\retex_sitrep_AAAAMMJJ.csv contient une ligne par message : horodate, tour, séquence, auteur, humain, score, défauts, message tronqué. Il s'ouvre directement dans un tableur, et c'est le support du débriefing.

6.3 La boucle sortie → entrée

Le plugin retourne DEFAULT_RELEVE, qui est injecté dans le prompt du tour suivant. Le COD, qui lit ce bloc comme le reste de sa situation, réclame de lui-même l'élément manquant. Le stagiaire n'est pas corrigé par le formateur : il est corrigé par le trafic.

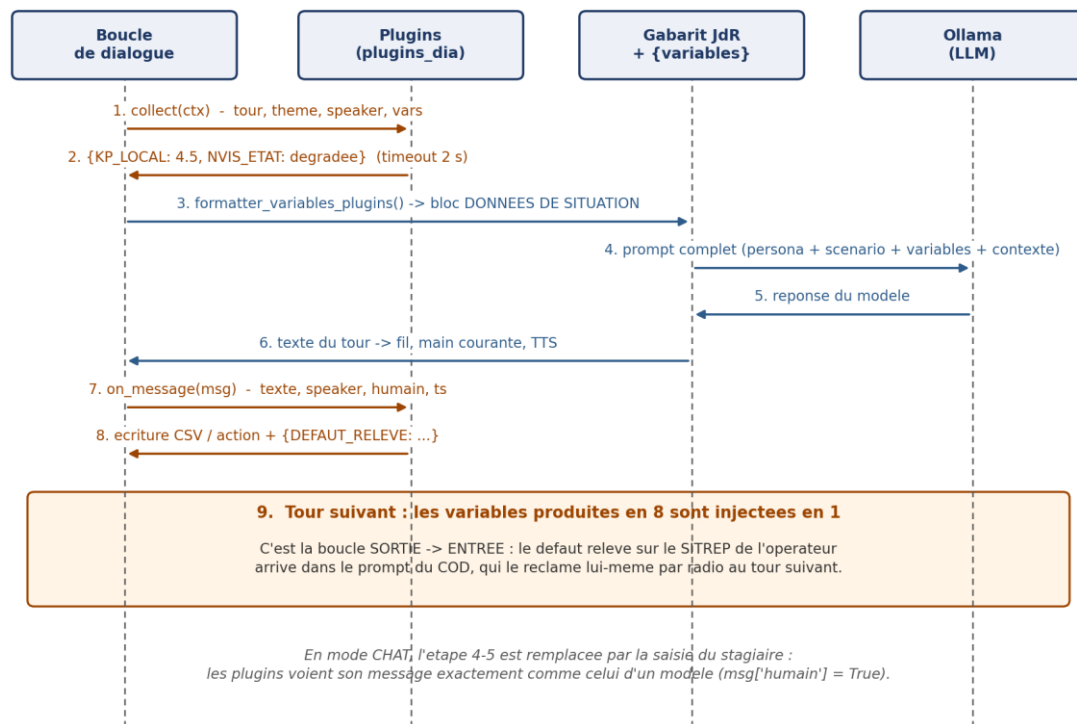


Figure 2 — Diagramme de séquence d'un tour de dialogue avec plugins

Choix de conception à connaître

Par défaut, le SCORE n'est PAS injecté : un COD qui commente la note d'un stagiaire détruit l'immersion en deux tours. Seul le défaut l'est, formulé comme une donnée de situation. Les deux comportements se règlent en tête du plugin par INJECTER_DEFAULT et INJECTER_SCORE.

6.4 Exploitation en RETEX

1. En fin de séance, cliquer sur Arrêter, puis sur le bouton SITREP pour produire le compte rendu PDF de la main courante.
2. Ouvrir logs\retex_sitrep_AAAAMMJJ.csv dans un tableur.
3. Trier par score croissant : les trois premières lignes donnent le plan du débriefing.
4. Comparer la colonne « défauts » entre le début et la fin de séance : c'est la progression du stagiaire, mesurée.
5. Archiver le CSV avec le preset et l'export du dialogue : l'exercice est rejouable à l'identique.

Exercice 6 — Workflow formateur complet

Décrivez, dans l'ordre, les opérations à effectuer pour préparer et conduire une séance HÉLIOS NOIR 26 en mode CHAT, couplée au Kp réel du jour et avec notation automatique. Depuis le poste éteint jusqu'au débriefing.

Indiquez en particulier : où vous récupérez le Kp, quel fichier vous éditez, quelles cases vous cochez, et ce que vous vérifiez avant de cliquer sur Démarrer.

→ Corrigé en annexe 11.5.

7. Actions, mesures et attentes (v1.11 à v1.11.5)

Jusqu'ici, un plugin observait : il apportait des données au dialogue et notait ce qui s'y disait. La v1.11 ferme la boucle dans l'autre sens — un LLM peut demander un acte ou une mesure, et attendre le résultat avant de poursuivre.

7.1 Du plugin qui observe au plugin qui agit

Trois besoins, un seul mécanisme :

- **Agir sur le monde** — allumer une lampe, enclencher un chauffage, basculer un relais, lancer une acquisition.
- **Relever une grandeur à la demande** — vitesse du vent, température, état de charge, dernier message reçu. Le modèle ne subit plus le rythme de la collecte : il demande.
- **Attendre** — ne rien dire de plus tant que la mesure n'est pas revenue, ou tant qu'un état n'est pas atteint. C'est ce qui distingue un dialogue qui raisonne sur des données d'un dialogue qui les invente.

Le mode recherche, enfin servi

Depuis la v1.11, le bloc {variables} est aussi injecté dans les gabarits du dialogue scientifique — Investigateur, Analyste, Modérateur, question initiale. Un dialogue de recherche sur un banc de mesure ou sur la propagation du jour n'est donc plus une simulation : les deux modèles discutent de valeurs réelles, et peuvent en demander de nouvelles.

7.2 Le contrat étendu

Deux fonctions s'ajoutent, reprises telles quelles du contrat des plugins IABrain :

```
def list_actions():
    # (action_id, libelle, description) - tuples a 3 elements, comme IABrain
    return [("chauffage_on", "Allumer le chauffage", "Fait monter la temperature"),
            ("mesure_vent", "Relever la vitesse du vent", "Anemometre du mat")]

def execute_action(action_id, params, ctx):
    # Retourne {NOM: valeur}, ou ({NOM: valeur}, ["avertissement"])
    if action_id == "chauffage_on":
        ...
    return {"CHAUFFAGE": "allume", "ACTION_RESULTAT": "chauffage enclenche"}
```

Le catalogue des actions déclarées est injecté dans le prompt, sous le bloc des variables, avec la syntaxe à employer. Un modèle ne peut donc demander que ce qui existe : une action inconnue est signalée dans le fil et n'a aucun effet.

7.3 Les trois directives

Le modèle écrit sa demande dans sa réponse, sur une ligne à elle seule :

```
ACTION: chauffage_on(zone=serre1)
MESURE: mesure_vent(mat=principal)
ATTENDRE: TEMPERATURE_C >= 30 delai=600
```

Directive	Ce que fait d-IA
ACTION:	Appelle execute_action(). Budget : 10 secondes — une action est un acte, pas une lecture. Le retour devient une variable.

Directive	Ce que fait d-IA
MESURE :	Strictement le même mécanisme. La distinction est sémantique : elle aide le modèle à formuler, et le journal à se relire.
ATTENDRE :	Suspend le dialogue et scrute la condition. Opérateurs : > >= < <= = != contient. Le délai est en secondes, plafonné par <code>plugins_attente_max_s</code> .

Pourquoi cette syntaxe, et pas du JSON

Le mot-clé est en MAJUSCULES, suivi de deux-points, en début de ligne, et la ligne ne contient rien d'autre. Une phrase de prose ne peut donc pas être prise pour une commande : « le COD demande une action : allumer le groupe électrogène » ne déclenche rien, et « Action: relancer(le moteur) » non plus — la casse ne correspond pas.

Un bloc JSON serait plus rigoureux pour le parseur, mais les modèles de 7 à 12 milliards de paramètres le cassent souvent, et il polluerait le trafic radio en mode exercice. La directive en clair se lit dans le fil, s'entend à la synthèse vocale, et se retrouve dans la main courante.

Trois directives au maximum par message : un modèle qui s'emballe ne peut pas emballer le dispositif.

7.4 Les trois modes d'exécution

Menu Plugins E/S ▼ → Actions demandées par les LLM. Le réglage est persisté et transporté par les presets.

Mode	Comportement	Quand l'utiliser
auto	Exécution directe, sans confirmation. C'est le mode par défaut.	Banc de mesure, domotique, acquisition météo — tout ce qui est réversible et sans risque.
confirmer	Une boîte de dialogue s'ouvre avant chaque action. Sans réponse au bout de 120 secondes, l'action est REFUSÉE.	Matériel sensible, première mise en service d'un plugin, démonstration devant un groupe.
off	La directive est journalisée dans le fil, rien n'est exécuté, et le catalogue n'est même pas proposé aux modèles.	EXERCICE FICTIF. Un COD simulé ne doit jamais actionner du matériel réel.

La règle à retenir pour un exercice

Un preset d'exercice doit déclarer `"plugins_actions_mode": "off"`. Le scénario HÉLIOS NOIR 26 fait dire au COD des phrases comme « faites le plein du groupe électrogène » : si un plugin de domotique déclarait une action de ce nom, le mode auto l'exécuterait. Le silence n'est pas une protection, la clé dans le preset en est une.

Inversement, pour une séance de recherche sur un banc, le mode auto est le seul utilisable : un dialogue qui demande une validation toutes les deux minutes n'avance pas.

7.5 L'attente, pas à pas

C'est la partie la plus délicate du mécanisme, parce qu'elle suspend la boucle de dialogue. Le schéma ci-dessous suit une demande complète : le modèle lance une acquisition, puis attend qu'elle aboutisse.

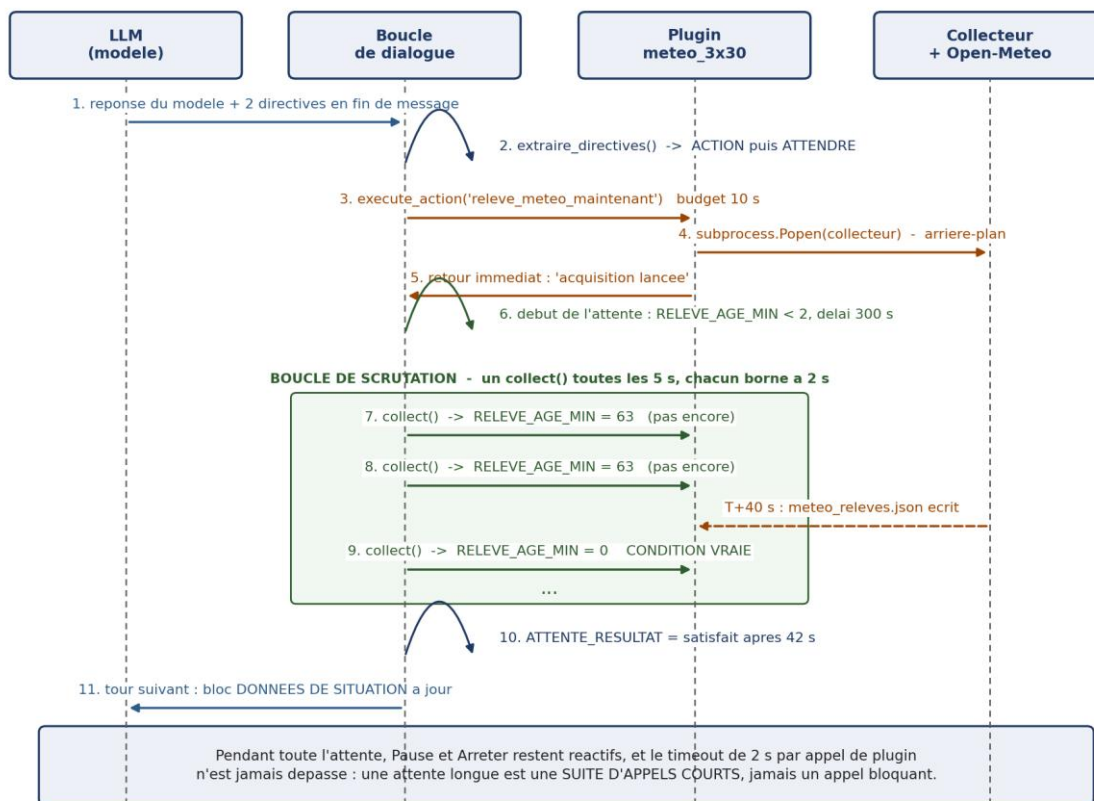


Figure 3 — Séquence d'une action suivie d'une attente

Trois propriétés à comprendre, parce qu'elles conditionnent l'écriture de vos plugins :

1. L'attente est une SUITE D'APPELS COURTS. d-IA rappelle `collect()` toutes les 5 secondes, et chaque appel reste borné à 2 secondes. Le timeout par plugin n'est jamais contourné : un plugin qui bloquerait pendant l'attente serait neutralisé comme n'importe quand.
2. L'attente est INTERRUPTIBLE. Pause suspend la scrutation, Arrêter y met fin. Le statut affiche la condition et le temps écoulé ; le fil reçoit un point toutes les 30 secondes, et un même avertissement de plugin n'est pas réémis avant 10 minutes — sans quoi un capteur muet noierait la main courante.
3. L'attente a TOUJOURS une issue, injectée dans `ATTENTE_RESULTAT` : « satisfait après 42 s », ou « NON satisfait : délai de 300 s dépassé (dernier relevé : `RELEVAGE_MIN = 63`) ». Le modèle sait donc s'il a obtenu sa mesure, et le RETEX aussi.

Ce dernier point est le plus important pédagogiquement : un échec est une information, pas un silence. Un opérateur qui lit « délai dépassé » dans le fil sait que la mesure n'est pas arrivée ; s'il ne lisait rien, il croirait que le modèle a inventé la suite.

7.6 Cas d'étude — meteo_3x30.py et la règle des 3 × 30

La règle des trois 30 caractérise un danger extrême de feu de végétation :

Critère	Seuil	Conséquence opérationnelle
Température de l'air	≥ 30 °C	Végétation desséchée, inflammation facile.
Vent moyen à 10 m	≥ 30 km/h	Propagation rapide, sautes de feu, front difficile à tenir.
Humidité relative	≤ 30 %	Combustibles fins très secs, reprises fréquentes.

Les trois réunis, c'est le signal d'une vigilance renforcée. Deux sur trois, c'est déjà une situation à surveiller. Le plugin injecte l'état de la règle ET le détail critère par critère, pour que les modèles puissent raisonner sur ce qui manque, pas seulement sur un verdict.

Une seule implémentation de la règle

La bibliothèque meteo_lib.py de TCQ calcule déjà cette règle pour la couche météo de la carte. Le plugin d-IA ne la réécrit pas : quand meteo_lib est accessible, il l'importe et appelle sa fonction rule_3x30(). Sinon, il applique une copie locale strictement identique.

```
def regle_3x30(temp_c, vent_kmh, hum_pct):
    if _meteo_lib is not None and hasattr(_meteo_lib, "rule_3x30"):
        try:
            return _meteo_lib.rule_3x30(temp_c, vent_kmh, hum_pct)
        except Exception:
            pass
    t = temp_c is not None and temp_c >= R3_TEMP
    w = vent_kmh is not None and vent_kmh >= R3_WIND
    h = hum_pct is not None and hum_pct <= R3_HUM
    return {"temp": bool(t), "wind": bool(w), "hum": bool(h),
           "count": int(t) + int(w) + int(h), "ok": bool(t and w and h)}
```

C'est une habitude à prendre : un plugin d-IA est une PORTE D'ENTRÉE vers vos outils existants, pas une deuxième version de leur logique. Le jour où un seuil change, il change à un seul endroit.

La séparation collecteur / plugin

Les données viennent du modèle AROME France HD de Météo-France (~1,5 km de maille), interrogé via Open-Meteo. Mais le plugin ne va jamais sur le réseau : deux fichiers se partagent le travail.

Fichier	Rôle
meteo_3x30_collecte.py	Hors d-IA. Interroge Open-Meteo en AROME France HD et écrit meteo_relevés.json (écriture atomique). Lancé par une tâche planifiée, par l'option --boucle 900, ou par une ACTION du LLM. En cas d'échec — réseau coupé, quota épuisé — il CONSERVE le relevé précédent.
meteo_3x30.py	Le plugin. Lit le fichier, calcule la règle, expose les variables, et déclare deux actions.

La chaîne complète, telle qu'un modèle l'écrit :

```
ACTION: releve_meteo_maintenant()      <- lance le collecteur, rend la main
ATTENDRE: RELEVÉ_AGE_MIN < 2 delai=300  <- attend la mesure fraîche
MESURE: mesure_meteo()                 <- relit et publie les valeurs
```

Et ce que les trois LLM lisent au tour suivant :

```
DONNEES DE SITUATION (relevés réels transmis au poste, à prendre en compte
dans ta réponse) :
REGLE_3X30 = ATTEINTE (3/3) - danger extrême de feu de végétation
[regle : T >= 30 C ET vent >= 30 km/h ET HR <= 30 %]
CRITERES_3X30 = temperature 34 C (seuil >= 30) OUI /
                vent 38 km/h (seuil >= 30) OUI /
                humidite 24 % (seuil <= 30) OUI
TEMPERATURE_C = 34.0
VENT_KMH = 38
RAFALES_KMH = 53
HUMIDITE_PCT = 24
LIEU_METEO = Nangeville (48.4, 2.7)
RELEVAGE_MIN = 1
ATTENTE_RESULTAT = RELEVAGE_MIN < 2 satisfait après 42 s
```

Le relevé simulé est signalé comme tel

Le collecteur accepte l'option --simuler 35,40,22 pour travailler hors ligne ou préparer une séance. Le fichier produit porte alors source: "simulation", et le plugin émet un avertissement visible dans le fil : « relevé SIMULÉ (jeu de test, pas une mesure AROME) ».

C'est la même exigence que pour un exercice : ce qui est fictif doit être marqué fictif. Un formateur qui projette une séance ne doit jamais avoir à se demander si la valeur affichée est une mesure.

7.7 Ce que cela change pour une séance

- **En exercice** : le scénario cesse d'être figé. Si le vent réel dépasse 30 km/h le jour de la séance, la règle des 3 × 30 s'affiche dans le prompt, et l'injecteur d'événements peut poser un départ de feu cohérent avec la météo du jour plutôt qu'avec le calendrier du scénario.
- **En recherche** : deux modèles peuvent conduire une campagne de mesures — demander un relevé, attendre, comparer, demander une variation, attendre encore. L'opérateur arbitre au lieu de saisir.
- **En veille** : une séance laissée en attente sur une condition (ATTENDRE: REGLE_3X30 contient ATTEINTE delai=1800) devient une surveillance, avec sa trace dans la main courante.

7.8 Où la mesure a-t-elle été faite ?

Ce qui suit n'est pas un exemple construit : c'est une séance du 13 septembre 2026, sur un poste ADRASEC 77, avec le plugin météo branché sur AROME France HD. L'opérateur pose au LLM1 une question simple — « Relève la météo à Melun et évalue la règle des 3 × 30 » — et le fil affiche ceci :

```
[Question au LLM1] Relève la météo à Melun et évalue la règle des 3 x 30.

[LLM1] ACTION: mesure_meteo
ATTENDRE: TEMPERATURE_C < 30 delai=60

[ACTION: mesure_meteo -> TEMPERATURE_C = 28.2, VENT_KMH = 10,
HUMIDITE_PCT = 36,
LIEU_METEO = Nangeville (48.4, 2.6999998),
RELEVAGE_MIN = 18]
[Attente satisfaite après 0 s - TEMPERATURE_C = 28.2]

[LLM1] Je ne peux pas répondre : il n'y a pas de données météorologiques
pour Melun dans les informations fournies.
```

Le modèle a raison, et son refus est la bonne réponse. Les valeurs affichées sont vraies — ce sont celles de Nangeville, relevées dix-huit minutes plus tôt — mais elles ne disent rien de Melun. Le défaut n'est ni dans le

scénario, ni dans le modèle, ni dans le prompt : il est dans le plugin, qui acceptait un paramètre lieu= sur releve_meteo_maintenant et l'ignorait sur mesure_meteo.

Notez au passage ce que le refus a coûté : un tour de dialogue, une acquisition inutile, et un doute. Si le modèle avait été moins scrupuleux — un modèle plus bavard aurait pu recopier 28,2 °C sous l'étiquette « Melun » — la séance aurait produit une information fausse, avec l'autorité que donne un chiffre affiché par un outil.

Une valeur sans lieu n'est pas une mesure

Une température est une grandeur relevée QUELQUE PART, à un MOMENT donné. Publier 28,2 °C sans dire où ni quand, c'est publier un nombre, pas une mesure.

Sur une règle de danger de feu, attribuer à une commune des valeurs relevées ailleurs n'est pas une approximation acceptable : c'est une information fausse. Et elle est d'autant plus dangereuse qu'elle sort d'un outil qui inspire confiance.

La règle vaut pour tout plugin d'entrée : niveau de batterie de QUEL relais, débit de QUELLE liaison, humidité de QUEL local.

La correction, livrée avec le plugin meteo_3x30 v1.6, tient en trois situations :

Ce que contient le fichier de relevés	Ce que fait mesure_meteo(lieu=melun)
Un relevé de Melun de moins de 2 minutes	Les valeurs sont rendues immédiatement, et MESURE_RESULTAT dit au modèle de répondre sans attendre.
Un relevé de Melun, mais plus ancien	Les valeurs sont rendues AVEC leur âge — elles sont vraies, seulement datées — et une acquisition est lancée, que le modèle peut attendre.
Un relevé d'un AUTRE site	Aucune valeur n'est publiée. L'acquisition de Melun est lancée, un avertissement nomme les deux lieux, et l'action répond : attendre RELEVAGE_MIN < 2.

Le fil de la même séance, après correction, devient :

```
[Plugin Meteo 3x30 - avertissement : le releve disponible concerne
Nangeville et non Melun (prefecture 77) : ses valeurs ne valent pas
pour Melun (prefecture 77)]
[ACTION: mesure_meteo(lieu=melun) -> MESURE_RESULTAT = acquisition AROME
lancee sur Melun - ATTENDRE: RELEVAGE_MIN < 2 delai=300,
LIEU_DEMANDE = Melun (prefecture 77), ACQUISITION = en cours]
[Attente de RELEVAGE_MIN < 2 - delai max 300 s]
[Attente satisfaite apres 5 s - RELEVAGE_MIN = 0]
```

L'avertissement est aussi important que la correction : il laisse dans la main courante la trace de ce qui a été écarté, et pourquoi. Un plugin qui corrige en silence apprend à l'opérateur à ne pas regarder.

Nommer les lieux à la place du modèle

Un modèle de langage ne connaît pas les coordonnées d'une commune — et surtout, il ne doit pas les inventer. On ne lui demande donc jamais une latitude : on lui donne une liste de sites nommés, qu'il lit dans les DONNÉES DE SITUATION, et le plugin résout les coordonnées lui-même :

```
SITES = {
    "nangeville": (48.40, 2.70, "Nangeville"),
    "melun": (48.61, 2.68, "Melun (prefecture 77)"),
    "fontainebleau": (48.40, 2.70, "Fontainebleau - foret"),
    "provins": (48.56, 3.30, "Provins"),
```

```
"coulommiers": (48.81, 3.09, "Coulommiers"),
"nemours": (48.27, 2.69, "Nemours"),
}
SITE_DEFAULT = "nageville"
```

La liste est republiée à chaque tour dans la variable `SITES_METEO` : le modèle sait donc toujours ce qu'il a le droit de demander. Un nom inconnu produit un avertissement qui rappelle la liste, et non un relevé silencieusement attribué au site par défaut. Adapter le plugin à votre département, c'est éditer ces six lignes — rien d'autre.

Variable	Ce qu'elle dit
LIEU_DEMANDE	Le site visé par la dernière action de mesure — ce que l'on a demandé.
LIEU_METEO	Le lieu du relevé effectivement lu — ce que l'on a obtenu.
RELEVÉ_AGE_MIN	L'âge de ce relevé, en minutes.
SITES_METEO	Les sites que le plugin sait relever.

Deux variables pour un seul lieu, cela peut sembler redondant. C'est au contraire le cœur du correctif : tant qu'elles sont distinctes, un opérateur qui relit le fil voit immédiatement l'écart entre la demande et la mesure. Une variable unique l'aurait masqué.

Une attente qui ne peut pas aboutir

Le correctif de la règle de lieu a immédiatement produit un second incident, dans la même séance. La mesure de Melun réussit, puis le dialogue se fige dix minutes :

```
[ACTION: mesure_meteo(lieu=melun) -> TEMPERATURE_C = 24.3,
VENT_KMH = 20.1, HUMIDITE_PCT = 59, REGLE_3X30 = 0 critere(s) sur 3,
LIEU_METEO = Melun (prefecture 77), RELEVÉ_AGE_MIN = 16,
LIEU_DEMANDE = Melun (prefecture 77)]
[Attente de RELEVÉ_AGE_MIN < 2 - delai max 600 s]
[Attente de RELEVÉ_AGE_MIN < 2 - 30 s ecoules, RELEVÉ_AGE_MIN = 17]
[Attente de RELEVÉ_AGE_MIN < 2 - 300 s ecoules, RELEVÉ_AGE_MIN = 21]
[Attente de RELEVÉ_AGE_MIN < 2 - DELAI DEPASSE (600 s), RELEVÉ_AGE_MIN = 26]

[LLM1] La meteo a Melun a ete relevee et la regle des 3 x 30 n'est pas
satisfaite car le critere d'age minimal de releve est depasse.
```

Regardez la première ligne : tout ce que l'opérateur avait demandé s'y trouve déjà. La température, le vent, l'humidité, le lieu, la règle des 3 × 30 évaluée. Le relevé de Melun datait de seize minutes ; le plugin l'avait jugé assez frais et n'avait donc lancé aucune acquisition.

Mais le modèle, lui, avait écrit `ATTENDRE: RELEVÉ_AGE_MIN < 2`. Il scrutait donc un compteur d'ancienneté que rien ne pouvait faire redescendre : à chaque scrutation, il montait. Dix minutes de séance perdues, puis une réponse qui annonce un échec au lieu de la météo qu'il avait sous les yeux — et qui confond au passage le délai d'attente avec la règle des 3 × 30.

Toute attente doit pouvoir aboutir

Une action qui invite le modèle à attendre doit GARANTIR que quelque chose va faire bouger la variable. Sinon, elle doit lui dire explicitement qu'il n'y a rien à attendre.

C'est au plugin de trancher, parce que lui seul sait s'il a déclenché quelque chose. Le moteur, lui, ne peut que constater l'expiration du délai.

Trois corrections, à trois niveaux — et c'est la répartition qui compte plus que chacune d'elles :

- **Dans le retour de l'action** (plugin v1.7) : MESURE_RESULTAT dit désormais dans lequel des deux cas on se trouve — « acquisition en cours, attendre RELEVÉ_AGE_MIN < 2 », ou « valeurs déjà à jour, répondre maintenant, ne pas attendre ».
- **Dans le catalogue** : la description de l'action ne prescrit plus une attente systématique, elle prescrit de LIRE MESURE_RESULTAT d'abord. Une description d'action est une consigne, pas une documentation.
- **Dans le seuil** : AGE_FRAIS_MIN passe de 20 minutes à 2. Une acquisition de plus coûte une requête réseau ; une attente sans objet coûte dix minutes de séance. Le compromis n'est pas symétrique.

Retenez surtout le raisonnement, car il se transpose à n'importe quel plugin d'action. Dès que vous écrivez une action qui déclenche quelque chose de lent, posez-vous la question dans cet ordre : ai-je vraiment déclenché quelque chose ? quelle variable va bouger ? et que se passe-t-il si je n'ai rien déclenché ? Si la réponse à la dernière question est « le modèle attendra pour rien », il manque une phrase dans votre retour d'action.

Côté opérateur, une habitude limite les dégâts : le paramètre `delai` n'est pas une formalité. Une attente de 600 secondes sur un compteur figé, c'est dix minutes de séance ; la même erreur avec `delai=60` se solde par une minute et un message clair. Commencez petit, allongez quand vous savez ce que vous attendez.

7.9 La fidélité d'une valeur publiée

Dans la même séance, une fois la météo de Melun obtenue, le LLM1 rédige son bulletin :

```
[LLM1] La meteo a Melun est la suivante :
Temperature : 24,3 C
Vent : 20,1 km/h (rafales jusqu'a 38 km/h)
Humidite : 59 %
La regle des 3x30 n'est pas remplie.
```

Tout est juste, sauf un détail : le plugin avait publié `VENT_KMH = 20`. La décimale n'existait nulle part — ni dans le relevé AROME tel que le plugin le publiait, ni dans le fil. Le modèle l'a ajoutée.

Ce n'est pas une hallucination spectaculaire, et c'est bien ce qui la rend instructive : elle est petite, plausible, et elle passe inaperçue. L'origine est de notre côté. Le plugin arrondissait le vent à l'entier (`f"{v:.0f}"`) alors qu'AROME le donne avec une décimale ; nous avons retiré une information, et le modèle a comblé le vide.

Publiez la valeur de la source, avec sa précision

Un arrondi est une information perdue. Un modèle de langage comble volontiers ce qu'on lui retire, et il le fait de façon crédible.

Corollaire : n'inventez pas non plus de précision. Si le capteur donne 24 °C, publiez 24 et non 24,0 — vous suggèreriez une exactitude qui n'existe pas.

Et publiez toujours ce qui qualifie la valeur : son lieu, son âge, sa source. Une valeur nue est une valeur invérifiable.

Le même réflexe se manifeste sur les seuils, et il est plus insidieux. Séance suivante, mesure de Melun obtenue, le LLM1 conclut correctement que la règle n'est pas remplie — puis il se justifie :

```
[LLM1] La regle des 3 x 30 n'est pas respectee car il n'y a pas
3 criteres sur 3 qui sont satisfaits : la temperature est
inferieure a 30 C, le vent est inferieur a 30 km/h et
l'humidite est inferieure a 80 %.
```

Les deux premiers seuils sont les bons. Le troisième n'existe nulle part — ni dans le plugin, ni dans le prompt, ni dans la règle des 3×30 , où le critère d'humidité est $HR \leq 30 \%$. Le plugin publiait « humidite 62 % non » : la valeur et le verdict, mais pas le critère. Le modèle a donc fabriqué un seuil pour justifier une conclusion par ailleurs exacte.

C'est ce décalage qui doit retenir l'attention : le verdict était juste, seul le raisonnement affiché était faux. En formation, c'est pourtant le raisonnement que le stagiaire lit et retient. Un verdict juste obtenu par un seuil inventé est une leçon fautive — et elle est d'autant plus difficile à rattraper que rien, dans le fil, ne signale l'erreur.

Depuis la v1.8, le plugin publie donc la valeur ET son critère, côte à côte :

```
REGLE_3X30 = 0 critere(s) sur 3
             [regle : T >= 30 C ET vent >= 30 km/h ET HR <= 30 %]
CRITERES_3X30 = temperature 23.9 C (seuil >= 30) non /
                vent 15.2 km/h (seuil >= 30) non /
                humidite 62 % (seuil <= 30) non
```

Trois fois le même mécanisme

La décimale du vent, le lieu du relevé, le seuil d'humidité : à chaque fois, le modèle a comblé ce que nous ne lui donnions pas, et il l'a fait de façon plausible.

Ce n'est pas un défaut du modèle : c'est une propriété des modèles de langage. Ils produisent ce qui manque. La seule parade est de ne rien laisser manquer.

Règle de conception : une variable de situation doit se suffire à elle-même. Une valeur, son unité, son lieu, son âge, et le critère selon lequel on la juge.

7.10 Ce que l'affichage ne doit pas cacher

Troisième enseignement de la même séance, et le plus discret. Dans le fil, la directive écrite par le modèle s'affichait ainsi :

```
[LLM1] ATTENDRE: RELEVAGEMIN < 2 delai=300
```

La ligne suivante du journal, elle, disait « Attente de RELEVAGE_MIN < 2 ». Le moteur avait donc lu le bon nom : la directive s'est exécutée correctement, l'attente a été satisfaite, rien n'a dysfonctionné. Seul l'affichage mentait — le rendu Markdown de la zone de conversation traitait le souligné comme l'astérisque et ouvrait une mise en italique au milieu du mot, faisant disparaître les deux soulignés.

C'est un défaut d'apparence, et en formation c'est précisément ce qui le rend sérieux : le stagiaire qui recopie la directive depuis la transcription projetée obtient un nom de variable faux, donc une attente qui n'aboutira jamais — et il cherchera l'erreur dans son plugin. Corrigé dans d-IA v1.11.2 : le souligné n'ouvre plus une emphase qu'en bordure de mot, la règle de CommonMark, et déjà celle qu'appliquait le nettoyage pour la synthèse vocale depuis la v1.4.

Pour l'auteur de plugin, deux conséquences

Vos noms de variables ne sont pas seulement lus par des modèles : ils sont lus, projetés et RECOPIÉS par des humains. Un affichage infidèle produit des directives fausses.

Si vous exportez vos variables ailleurs — main courante, tableau de RETEX, page web, message LXMLF — vérifiez que la chaîne de rendu ne les abîme pas. Le SCREAMING_SNAKE_CASE traverse mal les moteurs Markdown.

Le balisage que l'on retire ne doit jamais être du contenu

Le souligné avait en réalité DEUX bourreaux, et le second était plus grave que le premier. Au-delà de l'affichage, d-IA assainissait aussi les valeurs rendues par les plugins avant de les écrire dans le prompt — une précaution raisonnable, puisque les modèles imitent le balisage qu'ils lisent. La règle était cependant trop large :

```
# jusqu'à la v1.11.2
txt = re.sub(r"[^*_#>|]+", "", txt)
```

Deux de ces caractères ne sont pas du balisage, mais du contenu :

- **Le souligné** : tous les noms de variables sont en snake_case. « acquisition en cours, attendre RELEVÉ_AGE_MIN < 2 » arrivait au modèle sous la forme « attendre RELEVÉ_AGE_MIN < 2 ». Le modèle lisait donc, dans le prompt lui-même, un nom de variable qui n'existe pas — et le recopiait consciencieusement dans une directive impossible à satisfaire.
- **Le chevron** : c'est un opérateur. Le rappel de règle publié par le plugin, « T >= 30 C ET vent >= 30 km/h », devenait « T = 30 C ET vent = 30 km/h ». Un seuil minimal transformé en égalité n'est pas une coquille : c'est un contresens sur une règle de vigilance.

Corrigé en v1.11.3 : seuls l'accent grave, l'astérisque et la barre verticale sont retirés partout ; le chevron et le dièse ne le sont plus qu'en DÉBUT de valeur, là — et seulement là — où ils ouvriraient une citation ou un titre. Le souligné n'est plus touché du tout, l'affichage ne le mangeant plus depuis la v1.11.2.

La question à se poser avant de filtrer

Un filtre sur du texte doit se demander, pour chaque caractère retiré : ce caractère peut-il porter du SENS dans ce que mes plugins publieront ?

Pour l'accent grave, l'astérisque et la barre verticale, la réponse est non : ce sont des marques de mise en forme. Pour le souligné, le chevron et le dièse, la réponse est oui : identifiants, opérateurs, numéros. La position tranche alors mieux que le caractère — en début de ligne, pas ailleurs.

Un filtre trop large ne produit pas une erreur visible : il produit une valeur plausible et fausse. C'est la pire des deux.

7.11 Bien formuler la question au LLM1

Depuis la v1.11.1, le bouton « Question au LLM1 » voit les plugins : le bloc DONNÉES DE SITUATION et le catalogue d'actions sont joints à la question, les directives de la réponse sont exécutées, et d-IA réinterroge une fois le modèle avec les données obtenues. La question type, pour une mesure météo, tient en quatre lignes :

```
Releve la meteo a Melun et evalue la regle des 3 x 30.
Utilise : ACTION: mesure_meteo(lieu=melun)
puis ATTENDRE: RELEVÉ_AGE_MIN < 2 delai=300
Ne donne aucune valeur qui ne figure pas dans les DONNEES DE SITUATION.
```

Chaque ligne a sa raison d'être :

- **La première** énonce le besoin en langage naturel — c'est elle que le modèle reformulera dans sa réponse finale.
- **La deuxième** nomme l'action ET le lieu. Sans le lieu, vous retombez exactement sur le cas du 7.8.
- **La troisième** impose l'attente. Sans elle, le modèle commenterait le relevé précédent, puisque l'acquisition n'est pas instantanée.

- **La quatrième** est la clause de non-invention. Elle a un effet mesurable sur les petits modèles locaux, et aucun coût.

Un dernier point observé en séance, utile à connaître avant de projeter un exercice. Les modèles locaux de petite taille inventent volontiers des noms d'action inspirés de la question :

```
[MESURE: temperature - action inconnue, ignoree]
[MESURE: vent - action inconnue, ignoree]
[MESURE: humidite - action inconnue, ignoree]
```

Le catalogue fait autorité : ces trois directives ont été rejetées sans aucun effet, ce qui est le comportement voulu. Mais chacune a consommé l'une des trois directives autorisées par message. C'est pourquoi le catalogue du plugin v1.6 énonce désormais explicitement que `mesure_meteo` fournit ENSEMBLE la température, le vent, l'humidité et la règle 3 × 30 : une description d'action ne sert pas seulement à documenter, elle sert à empêcher le modèle d'imaginer les actions qui manquent.

Exercice 9 — Une action et son attente

Un mât de mesure expose un anémomètre et un enregistreur. Vous voulez qu'un LLM puisse déclencher un enregistrement de 10 minutes, puis attendre qu'il soit terminé avant de commenter les résultats.

- Quelles actions déclarez-vous dans `list_actions()`, et que retourne chacune ?
- Quelle variable permet au LLM de savoir que l'enregistrement est terminé, et quelle directive ATTENDRE écrit-il ?
- Pourquoi `execute_action()` ne doit-elle SURTOUT PAS attendre elle-même la fin des 10 minutes ?
- Que se passe-t-il si l'enregistreur ne répond jamais ?

→ Corrigé en annexe 11.5.

Exercice 10 — Audit d'une directive et d'un mode

Pendant une séance d'exercice HÉLIOS NOIR 26 en mode CHAT, le fil affiche :

```
[ACTION: groupe_electrogene_on(site=EHPAD) -> ALIMENTATION = retable]
```

- Qu'est-ce qui a rendu cela possible, et en quoi est-ce un incident ?
- Quelles deux corrections apportez-vous, l'une dans le preset et l'autre dans d-IA ?
- Le formateur propose de renommer l'action en « `groupe_electrogene_simule_on` ». Pourquoi cette réponse est-elle insuffisante ?

→ Corrigé en annexe 11.5.

Exercice 11 — Le relevé d'à côté

Vous écrivez un plugin pour trois capteurs d'humidité d'un même bâtiment relais : cave, combles, local technique. Les trois écrivent dans un seul fichier de relevés, et vous exposez l'action `mesure_humidite(lieu=<capteur>)`.

- Quel défaut du plugin météo v1.5 risquez-vous de reproduire tel quel ?

- b) Quelles variables publiez-vous pour qu'un lecteur du fil sache toujours de quel capteur vient la valeur affichée ?
- c) Le capteur demandé n'a pas été relevé depuis deux heures, les deux autres l'ont été il y a cinq minutes. Que rendez-vous ?
- d) Un formateur propose, « faute de mieux », de rendre la valeur du capteur le plus proche. Que lui répondez-vous ?

→ Corrigé en annexe 11.5.

Exercice 12 — Trois chiffres et un affichage

Pendant une séance, le fil affiche les deux lignes suivantes, produites par votre plugin :

```
MESURE_RESULTAT = releve de la cave - ATTENDRE: MESUREAGEMIN < 5
```

```
HUMIDITE = 62
```

- a) L'un des deux noms affichés est faux. Lequel, pourquoi, et la directive s'exécutera-t-elle quand même ?
- b) Le capteur a mesuré 61,8 %. Pourquoi publier 62 est-il un mauvais choix, alors que l'écart est négligeable ?
- c) Quelle information manque-t-il à ces deux lignes pour qu'un opérateur puisse s'y fier ?

→ Corrigé en annexe 11.5.

8. Cas d'étude 3 — chronogramme.py : un plugin qui conduit la séance

Les deux premiers cas d'étude rapportaient le monde à la simulation : la propagation, la météo. Le troisième fait l'inverse — il ne mesure rien, il décide de ce qui arrive et à qui. C'est le premier plugin qui prend la main sur le déroulement, et il pose des questions de conception que les précédents n'avaient pas posées.

8.1 Le problème : une animation qui improvise

Jusqu'à la v1.11.11, la cellule d'animation de d-IA (LLM3) tirait un événement au sort dans un répertoire, tous les K tours. Le résultat était plausible, parfois excellent — et pédagogiquement creux. Rien ne reliait un événement à un objectif, donc rien n'était évaluable en fin de séance.

Le guide méthodologique de la Direction de la Sécurité Civile sur les exercices Cadre et Terrain (2011) décrit une méthode inverse. Le tableau d'animation — le chronogramme — se remplit de la droite vers la gauche :

1. On écrit d'abord la RÉACTION ATTENDUE des joueurs, celle qui découle d'un objectif.
2. On choisit le joueur qui doit la produire : ce sera le récepteur.
3. On choisit l'animateur qui va la déclencher : ce sera l'émetteur.
4. On rédige alors seulement l'ÉVÉNEMENT, mot à mot, dans le langage métier de cet animateur.
5. On décide enfin du vecteur de communication et du groupe horaire.

À retenir

Écrire d'abord l'événement, c'est écrire une histoire. Écrire d'abord la réaction attendue, c'est écrire un exercice.

Le plugin ne fait rien d'autre que rendre ce tableau exécutable.

8.2 La donnée d'entrée : un CSV, pas du code

Le chronogramme est écrit par le formateur, dans un tableur, avant la séance. Le plugin ne fait que le lire. C'est un choix de conception délibéré : celui qui monte l'exercice n'est pas celui qui programme, et un exercice se relit à plusieurs.

```
#EXERCICE=HELIOS NOIR 26 - sequence 7
#DEBEX=2026-07-11 14:00
#MINUTES_PAR_TOUR=5
groupe_horaire;vecteur;emetteur;recepteur;evenement;reaction_attendue;detection;relance
H+5;VARA FM;ANIBAS jouant l'EHPAD;LLM2;EXERCICE. Climatisation a l'arret
depuis 11h, deux residents en hyperthermie.;SITREP formate et demande
chiffree de carburant;carburant|litres|evacuation;EXERCICE. La directrice
rappelle, un troisieme resident est en hyperthermie.
```

Colonne	Rôle
groupe_horaire	H-15, H, H+5, H+120 — ou RESERVE pour un incident hors chronologie.
vecteur	VARA FM, HF NVIS, téléphone, radio phonie. Le guide insiste : les moyens doivent être ceux que les joueurs utiliseraient réellement.
emetteur	ANIBAS jouant l'EHPAD, ANIHOUT jouant le COZ, DIRANIM.
recepteur	LLM1, LLM2, TOUS, ou un libellé de rôle du preset.
evenement	Le message, rédigé mot à mot, tel qu'il sera lu.
reaction_attendue	JAMAIS publiée. Voir 8.4.

Colonne	Rôle
detection	Motifs séparés par une barre verticale, cherchés dans les messages des joueurs.
relance	L'inject de rattrapage, quand la réaction ne vient pas. Voir 8.6.

8.3 Une horloge de jeu, et pourquoi elle règle un vieux problème

Avant ce plugin, les modèles inventaient leurs horodates. Ils le faisaient bien — 11h37, 14h20 — mais sans cohérence entre eux, et sans rapport avec la durée réelle de la séance. Une main courante dont les heures ne s'enchaînent pas n'est pas exploitable en RETEX.

Le plugin convertit les prises de parole en minutes de jeu et publie l'heure :

```
HEURE_JEU = 11/07 14h20 (H+20)
```

Les deux formes sont là : l'heure de jeu absolue, et le groupe horaire relatif au DEBEX. C'est la double colonne que le guide demande. Les personas n'ont plus qu'à recevoir une consigne :

```
- Les DONNEES DE SITUATION portent HEURE_JEU, l'heure de l'exercice : tu
horodates tes messages avec elle, jamais avec l'heure réelle.
```

La cadence par défaut est de 5 minutes de jeu par prise de parole. Elle se change à chaud, sans toucher au fichier, par une variable de session — voir 8.7.

8.4 Le point délicat : la réaction attendue ne doit jamais sortir

Le fichier contient, à côté de chaque événement, ce que les joueurs sont censés faire. Si cette colonne arrivait dans les DONNÉES DE SITUATION, les modèles la liraient et l'exécuteraient : l'exercice n'aurait plus aucune valeur. Le guide en fait une règle de conduite, « pas de divulgation », et ajoute qu'« un exercice faussé n'a aucun intérêt ».

Un plugin qui lit un fichier contenant à la fois du public et du confidentiel doit donc trier — et le tri doit être vérifiable :

1. La colonne `reaction_attendue` n'apparaît dans AUCUNE variable publiée.
2. Elle n'apparaît pas non plus dans les avertissements, qui s'affichent dans le fil.
3. Elle est écrite dans la main courante de RETEX, qui n'est lue par personne pendant la séance.
4. Un cas de test dédié déroule douze tours et vérifie qu'aucune sortie ne la contient.

La leçon, valable pour tout plugin

Dès qu'un plugin manipule une donnée que les modèles ne doivent pas voir, la séparation ne se contrôle pas à la relecture : elle s'écrit dans un test. Ici, le test est le seul moyen de garantir qu'aucune évolution ultérieure ne fera fuiter la colonne.

8.5 Un inject, un destinataire

Les variables de situation sont injectées dans le prompt des trois modèles. Un événement publié bêtement serait donc reçu par tout le monde à la fois — ce qui supprimerait précisément ce qu'un exercice cadre cherche à tester : la circulation de l'information dans le centre opérationnel.

Le plugin s'appuie sur une clé de `ctx` dont les deux premiers cas d'étude n'avaient pas eu l'usage :

```
def collect(ctx):
```

```
...
if not destinataire_est(ligne, ctx): # ctx["speaker"], ctx["role"]
    _ETAT["attente"] = ligne["n"] # on patiente
    return _borner(out), avertis
```

L'inject n'est publié qu'au tour du joueur désigné. L'autre partie ne l'a pas reçu ; elle l'apprendra si — et seulement si — le destinataire le relaie. Le guide donne d'ailleurs ce symptôme parmi ceux qu'un animateur doit relever : « ils ont eu plusieurs appels de joueurs différents pour la même demande de renseignements, ce qui démontre un manque de communication transverse au sein du CO ».

Un destinataire peut ne jamais prendre la parole — un rôle coupé, un mode CHAT où le stagiaire ne répond pas. Le plugin ne bloque pas pour autant : au bout de six tours, l'inject part à qui parle, et l'anomalie est signalée. Une règle qui ne prévoit pas son cas dégradé arrête la séance.

8.6 Pas de constat d'échec : la relance

La réaction attendue ne vient pas toujours. La tentation est de le noter pour le débriefing ; le guide dit le contraire :

Plutôt qu'annoncer en fin d'exercice une enumeration des erreurs commises, il est preferable de relancer l'evenement ou l'incident par un AUTRE VECTEUR de communication pour faire prendre conscience aux joueurs qu'ils ont rate quelque chose mais qu'ils ont encore le temps de reagir comme dans la realite.

Le plugin implémente cette règle. Il surveille les messages des joueurs par `on_message`, cherche les motifs de la colonne detection, et si rien n'est venu au bout de quatre tours, il publie le texte de la colonne relance, marqué « par un autre vecteur ».

Deux précautions de conception méritent d'être relevées, parce qu'elles se généralisent :

1. Seul un JOUEUR peut produire une réaction attendue. Un inject de l'animation qui contiendrait le motif ne vaut pas réaction — `on_message` ignore donc LLM3.
2. Le FINEX ne tombe pas tant qu'une relance reste possible : l'exercice ne se termine pas sur une réaction qu'on n'a pas laissé venir.

8.7 Le pupitre : piloter sans exposer d'action

Le formateur doit pouvoir avancer, ralentir, sortir un incident de réserve, prononcer le FINEX. La solution évidente serait de déclarer des actions dans `list_actions()`. Elle est mauvaise, et pour une raison de sûreté déjà rencontrée au chapitre 7.

Un preset d'exercice doit imposer `"plugins_actions_mode": "off"`. Si le pupitre passait par des actions, il serait donc indisponible précisément là où il sert. Pire : les actions figurent au catalogue injecté dans le prompt, et un COD simulé pourrait décider lui-même d'avancer le chronogramme de son propre exercice.

Le pupitre passe donc par les VARIABLES DE SESSION — celles que l'opérateur saisit à la main, qui priment sur celles des plugins et sont prises en compte immédiatement :

Variable de session	Effet
DIRANIM_AVANCE = 1	Pousse l'inject suivant. Chaque NOUVELLE valeur compte pour une avance : sans cette règle, une variable oubliée dans le pupitre déroulerait tout l'exercice.
DIRANIM_RESERVE = 2	Injecte l'incident de réserve n° 2.
DIRANIM_PAUSE = oui	Fige l'horloge de jeu.
MINUTES_PAR_TOUR = 10	Change la cadence.

Variable de session	Effet
DIRANIM_FINEX = oui	Prononce le FINEX.
CHRONOGRAMME = seq7	Change de chronogramme.

La seule action déclarée par le plugin, `chronogramme_etat`, est en lecture seule : elle rend l'heure de jeu et l'avancement, et n'injecte rien.

Le critère, pour vos propres plugins

Une capacité qui appartient au formateur passe par une variable de session. Une capacité qui appartient aux modèles passe par une action. Se tromper de canal, c'est soit priver l'opérateur de son pupitre, soit donner aux modèles une commande qui ne leur revient pas.

8.8 Se taire quand on n'a rien à dire

Un plugin déposé dans `plugins_dia\` est chargé pour TOUTES les séances. Or la plupart des séances n'ont pas de chronogramme : une séance de recherche, un jeu de rôle libre, un essai. Deux défauts guettent alors le plugin :

1. prendre la main sur une séance qui ne lui a rien demandé — et la terminer par un FINEX qui n'a aucun sens ;
2. répéter à chaque tour un avertissement « fichier absent » qui pollue le fil.

La réponse tient en deux règles. Le chronogramme est un choix EXPLICITE : sans `chronogramme.csv` et sans variable `CHRONOGRAMME`, le plugin ne publie rien du tout. Et l'avertissement n'est dit qu'UNE fois par fichier, pas à chaque appel.

C'est une propriété générale : un plugin qui n'a rien à apporter doit rendre un dictionnaire vide et se taire. Le chapitre 3 le disait déjà pour une source absente ; il vaut aussi pour un plugin entier hors de son emploi.

8.9 Ce que le chronogramme rend possible : évaluer

C'est le vrai apport, et il ne vient pas du déroulé mais du pointage. Le chronogramme portant les réactions attendues, le plugin peut confronter ce qui s'est dit à ce qu'on attendait, et écrire une main courante exploitable :

```
logs\retex_chronogramme_AAAAMMJJ.csv
horodate_reel ; heure_jeu ; tour ; nature ; groupe_horaire ; vecteur ;
emetteur ; recepteur ; evenement ; reaction_attendue ; pointee ; extrait_joueur
```

La colonne « nature » distingue un inject, une relance, un incident de réserve et une réaction. La colonne « pointee » dit oui ou non. L'extrait donne le message du joueur qui a produit la réaction — c'est lui qu'on relit en débriefing.

Au FINEX, le bilan s'affiche dans le fil :

```
[Plugin Chronogramme : FINEX - 9 inject(s), 6 reaction(s) attendue(s)
obtenue(s) sur 9 pointable(s), 2 relance(s).]
```

Avant ce plugin, une séance produisait un dialogue, et le débriefing ne pouvait être qu'une appréciation littéraire. C'est la différence entre un exercice qui se joue et un exercice qui s'évalue.

8.10 Exercices

Exercice 13 — La colonne qui ne doit pas sortir

Vous reprenez `chronogramme.py` pour votre propre exercice et vous ajoutez une colonne « `objectif_vise` », destinée au seul RETEX : elle rappelle quel objectif de l'exercice chaque ligne sert.

- Cette colonne doit-elle apparaître dans les variables publiées ? Justifiez par la doctrine.
- Un collègue propose de la publier « puisque les joueurs connaissent de toute façon les objectifs de l'exercice, on les leur a lus au DEBEX ». Que lui répondez-vous ?
- Vous l'écrivez donc dans la main courante seulement. Quel contrôle ajoutez-vous pour être sûr qu'une évolution ultérieure ne la fera pas fuiter ?
- Le plugin publie déjà `INJECT_DE` (« ANIBAS jouant l'EHPAD »). N'est-ce pas, en soi, une divulgation — le joueur apprend que son interlocuteur est un animateur ?

→ Corrigé en annexe 11.5.

Exercice 14 — Un pupitre mal placé

Un formateur écrit un plugin de conduite de séance pour un exercice SATER. Il déclare trois actions : `passer_a_la_phase_suivante`, `injecter_incident` et `terminer_exercice`. Il les essaie, tout marche. Puis il charge son preset d'exercice, et plus rien ne répond.

- Que s'est-il passé ?
- Il envisage de retirer `plugins_actions_mode` du preset pour retrouver son pupitre. Que se passerait-il en séance ?
- Quelle est la bonne solution, et sur quel critère la choisit-on ?
- Il garde malgré tout une action `etat_exercice`, en lecture seule. Est-ce contradictoire avec ce qui précède ?

→ Corrigé en annexe 11.5.

9. Écrire son propre plugin

8.1 Squelette minimal

Le fichier `exemple_minimal.py` livré dans `plugins_dia\` est fait pour être copié :

```
# -*- coding: utf-8 -*-
"""mon_plugin.py - <objet en une ligne>"""

import datetime

PLUGIN_NAME = "Mon plugin"
PLUGIN_VERSION = "1.0"

_ETAT = {"messages": 0}          # etat local, persiste pendant la session

def list_variables():
    return ["HEURE_EXERCICE", "Heure locale, pour horodater le trafic"]

def collect(ctx):
    return {"HEURE_EXERCICE": datetime.datetime.now().strftime("%Hh%M")}

def on_message(msg, ctx):
    _ETAT["messages"] += 1
    return {"MESSAGES_ECHANGES": _ETAT["messages"]}

if __name__ == "__main__":      # autotest hors d-IA
    print(collect({"tour": 1, "theme": "test", "speaker": "LLM1", "vars": {}}))
    print(on_message({"speaker": "LLM2", "texte": "essai", "humain": True}, {}))
```

8.2 Méthode en six étapes

1. Esquisser le contrat : quelles variables je produis, à quel moment, et qui doit les lire — le COD, l'opérateur, ou l'injecteur ?
2. Écrire la logique métier pure, sans aucune dépendance à d-IA : des fonctions qui prennent des valeurs et retournent des valeurs. Les tester dans un interpréteur.
3. Ajouter la couche d-IA : `list_variables`, `collect`, `on_message`. Aucune logique dans ces fonctions, seulement de l'aiguillage.
4. Écrire un autotest sous `if __name__ == "__main__"` et le lancer : `python mon_plugin.py`. Le plugin doit fonctionner seul.
5. Déposer le fichier dans `plugins_dia\`, recharger les plugins, ouvrir le Diagnostic et vérifier la ligne [OK] et les variables déclarées.
6. Lancer une séance de cinq minutes en mode 3 LLM et lire le bloc **DONNEES DE SITUATION** dans le diagnostic : ce que vous y voyez est exactement ce que les modèles lisent.

8.3 Bonnes pratiques

- **Lire, ne pas interroger.** Un plugin lit un fichier déjà écrit par un autre programme. Ce n'est pas lui qui interroge un service, un capteur ou un site web : c'est un service tiers qui écrit le fichier, à son rythme.
- **Une variable, une valeur.** Pas de phrase, pas de paragraphe, pas de rapport. Si vous avez besoin de trois phrases, vous avez besoin d'un persona, pas d'une variable.
- **Nommer en opérationnel.** `AUTONOMIE_PCO` parle à un opérateur ; `VAL_3` ne parle à personne — ni au formateur, ni au modèle.

- **Écrire pour le modèle.** « dégradée (absorption, liaisons HF aleatoires) » produit un meilleur trafic que « 4.5 ». Une valeur brute oblige le modèle à l'interpréter ; une valeur qualifiée lui dit quoi en faire.
- **Garder l'état local.** Un dictionnaire de module suffit pour un compteur. Le module reste chargé toute la session.
- **Journaliser hors du fil.** Pour déboguer, écrivez dans un fichier, pas dans une variable : le fil de conversation est un espace d'exercice, pas une console.

8.4 Pièges classiques

Symptôme	Cause et remède
Le plugin est chargé mais rien n'est injecté	La case « Activer les plugins et variables de situation » est décochée, ou collect() retourne un dictionnaire vide. Vérifier dans le Diagnostic les « Variables au dernier tour ».
Le plugin est désactivé en pleine séance	Deux timeouts. Le plugin fait un appel lent (réseau, port série). Déporter l'acquisition dans un programme tiers qui écrit un fichier.
Les modèles ignorent la variable	Nom obscur ou valeur brute. Qualifier la valeur et nommer la variable en langage opérationnel. Au besoin, ajouter une ligne au persona : « Tu consultes les DONNEES DE SITUATION avant de répondre. »
Les réponses deviennent bavardes	Le bloc est trop gros. 12 variables maximum, et 3 ou 4 suffisent presque toujours.
Des accents ou du markdown apparaissent dans le trafic	Les valeurs sont nettoyées du markdown mais pas des accents : écrivez vos valeurs sans accent, comme le reste du preset.
Le CSV n'est pas écrit	Fichier ouvert dans un tableur, ou dossier absent. Le plugin ignore l'échec pour ne pas casser la séance : vérifier le chemin dans le Diagnostic.

Exercice 7 — Compléter la notation des SITREP

Ajoutez à sitrep_scoring.py un septième critère : « fréquence ou moyen de liaison annoncé » (par exemple « en VARA FM », « sur 145.250 », « via QO-100 »), valant 2 points, et ramenez le total à 20 en retirant 2 points ailleurs.

Écrivez l'expression régulière, la modification de _noter(), et dites ce qu'il faut changer dans l'en-tête du CSV — et pourquoi c'est un problème si des séances précédentes ont déjà écrit dans le fichier du jour.

→ Corrigé en annexe 11.5.

Exercice 8 — Plugin « main courante TCQ » (projet)

Écrivez un plugin complet tcq_maincourante.py qui lit le dernier message reçu par TCQ (supposé disponible dans un fichier maincourante.json écrit par TCQ, contenant une liste d'objets {ts, de, vers, texte}) et qui produit :

DERNIER_MSG_RECU (expéditeur et début du texte, 100 caractères), MSG_EN_ATTENTE (nombre de messages non traités depuis le début de la séance), et LIAISON_ACTIVE (le moyen du dernier message).

Contraintes : aucune dépendance hors bibliothèque standard, repli silencieux si le fichier n'existe pas, avertissement si le fichier n'a pas bougé depuis plus de 30 minutes, et autotest fonctionnel.

→ Corrigé en annexe 11.5.

10. Tests

9.1 L'autotest local

Un plugin doit pouvoir se lancer seul, sans d-IA. C'est la première chose à écrire et la dernière à supprimer :

```
if __name__ == "__main__":
    ctx = {"tour": 3, "theme": "Sequence 2", "speaker": "LLM1",
          "role": "COD", "sujet": "EXERCICE", "mode_jdr": True,
          "labels": {}, "vars": {}, "historique_court": []}
    print("collect      :", collect(ctx))
    msg = {"tour": 4, "speaker": "LLM2", "role": "Operateur",
          "texte": "EXERCICE - F1XYZ, 14h20. Accuse reception. EXERCICE",
          "theme": "Sequence 2", "ts": "2026-09-13T14:20:00", "humain": True}
    print("on_message  :", on_message(msg, ctx))
```

Lancez-le avec `python mon_plugin.py`. Si cette commande échoue, inutile de déposer le fichier dans `plugins_dia\`.

9.2 Tester les cas dégradés

Les trois quarts des défauts d'un plugin apparaissent hors du cas nominal. Testez systématiquement :

- Fichier de données absent — le plugin doit retourner {} sans exception.
- Fichier présent mais malformé, ou en cours d'écriture par l'autre programme.
- Valeur présente mais du mauvais type (texte au lieu de nombre, null).
- Valeur aberrante ($K_p = 47$, autonomie négative) : que produit votre plugin ?
- Appel répété 50 fois d'affilée : votre plugin garde-t-il un état qui enfle indéfiniment ?

9.3 Ce que d-IA teste déjà pour vous

Inutile de vous protéger contre ce que le cœur gère : d-IA capture toute exception, borne la durée d'appel, assainit et plafonne vos valeurs, et n'injecte rien si vous ne produisez rien. La suite de tests livrée avec la v1.10 couvre le chargeur lui-même (fichier ignoré, syntaxe invalide, hook absent, exception, timeout puis désactivation, assainissement, plafonds, priorité des variables manuelles, injection dans le gabarit). Celle de la v1.11 y ajoute l'extraction des directives, le refus de confondre la prose avec une commande, le plafond de trois directives, le comparateur, l'exécution d'une action avec ses paramètres, l'attente satisfaite, l'attente expirée, et le mode off qui n'actionne rien.

Ce dont vous restez seul responsable : la justesse de vos données, la pertinence de vos noms de variables, et le fait de ne jamais inventer une valeur.

11. Annexes

11.1 Glossaire

Terme	Définition
Hook	Point d'accroche : fonction de votre plugin que d-IA appelle à un moment précis de la boucle de dialogue.
collect	Hook d'entrée, appelé avant chaque prise de parole.
on_message	Hook de sortie, appelé après chaque prise de parole.
ctx	Dictionnaire décrivant l'état de la simulation au moment de l'appel.
msg	Dictionnaire décrivant la prise de parole qui vient d'avoir lieu.
Variable de situation	Paire nom/valeur injectée dans le prompt des modèles sous le bloc DONNEES DE SITUATION.
Variable de session	Variable de situation saisie à la main par l'opérateur (équivalent du /set d'IAbrain).
Bloc {variables}	Emplacement, dans les gabarits de prompt JdR, où d-IA écrit les variables.
Timeout	Durée maximale d'un appel de plugin (2 s). Au-delà, le résultat est ignoré.
Preset	Fichier .diapreset.json décrivant un scénario complet : sujet, séquences, rôles, réglages.
Mode CHAT	Mode où l'opérateur humain joue le LLM2 à la place d'un modèle.
RETEX	Retour d'expérience conduit à l'issue de l'exercice.
Rappel de critère	Le seuil publié a cote de la valeur : sans lui, le modèle invente un seuil plausible pour justifier son verdict (v1.8 du plugin meteo).
Attente aboutissable	Une action qui invite à attendre doit garantir qu'une variable va bouger ; sinon elle doit dire qu'il n'y a rien à attendre (v1.11.2).
Règle de lieu	Un relevé appartient à un seul site : un plugin ne publie jamais, pour un lieu demandé, les valeurs mesurées ailleurs (v1.11.2).
Catalogue d'actions	Liste des actions déclarées par les plugins, injectée dans le prompt. Elle fait autorité : une action absente est rejetée.
AROME France HD	Modèle de prévision de Météo-France (~1,5 km), source des relevés du plugin meteo_3x30.
Exercice cadre	Exercice sur table, en salle de crise, sans engagement de moyens sur le terrain (guide DSC 2011).
Chronogramme	Tableau qui donne, ligne par ligne, tout le déroulement de l'exercice du DEBEX au FINEX.
Réaction attendue	Ce que les joueurs doivent produire à la réception d'un événement. Elle commande l'événement, jamais l'inverse — et elle ne leur est jamais montrée.
Inject	Événement remis à un joueur par l'animation, à un groupe horaire donné et par un vecteur donné.
DIRANIM	Directeur de l'animation : il déroule le chronogramme, recale le jeu, prononce le FINEX.
ANIBAS / ANIHAUT	Animation basse (échelons inférieurs : CODIS, mairies, terrain) et haute (échelons supérieurs non joués : COZ, COGIC).
DEBEX / FINEX	Début et fin de l'exercice. Le DEBEX est précédé d'un message de mise en ambiance qui teste les liaisons.

Terme	Définition
Pas de divulgation	Qui monte l'animation ne joue pas, et ne donne pas le scénario aux joueurs. « Un exercice faussé n'a aucun intérêt. »
Pas de constat d'échec	Une réaction qui ne vient pas se relance par un autre vecteur ; elle ne se reproche pas en débriefing.

11.2 Fichiers et emplacements

Chemin	Contenu
plugins_dia\	Les plugins. Créé automatiquement au premier démarrage.
plugins_dia\README.md	Le contrat d'interface, en version courte.
plugins_dia\propagation_relevés.json	Relevés lus par propagation_vars.py (gabarit auto-créé).
plugins_dia\meteo_relevés.json	Relevés météo écrits par meteo_3x30_collecte.py.
plugins_dia\meteo_3x30_collecte.py	Collecteur réseau (AROME). Options --simuler, --boucle, --sortie.
DIA_METEO_RELEVÉS	Variable d'environnement : partage d'un même fichier de relevés entre plusieurs installations.
plugins_dia\chronogramme.py	Le plugin qui déroule un chronogramme d'exercice cadre.
plugins_dia\chronogramme.csv	Le chronogramme par défaut. CHRONOGRAMME = <nom> en fait lire un autre.
plugins_dia\chronogramme_helios_noir_26.csv	Le chronogramme livré : séquence 7 d'HÉLIOS NOIR 26.
logs\retex_sitrep_AAAAMMJJ.csv	Notation produite par sitrep_scoring.py.
logs\retex_chronogramme_AAAAMMJJ.csv	Main courante de RETEX : injects, réactions attendues et leur pointage.
d-ia_setup.json	Configuration : plugins_actifs, session_vars.
d-IA_preset*.diapreset.json	Presets d'exercice ; transportent aussi plugins_actifs.
archives\	Sauvegardes horodatées des versions précédentes du source.

11.3 Versions et historique

Version	Date	Apport
1.2.0	mai 2026	Presets de sujet : le scénario devient un fichier partageable.
1.3.0	juin 2026	Premier bloc injecté depuis une source extérieure : {sources_web}.
1.7.0	juin 2026	Mode Jeu de Rôle et rôles personnalisables par preset (jdr_profil).
1.8.0	juin 2026	Mode CHAT : l'opérateur humain joue le LLM2.
1.9.0	juin 2026	Génération d'un preset depuis un PDF de scénario.
1.10.0	septembre 2026	Plugins d'entrées-sorties, bloc {variables}, variables de session.
1.11.0	septembre 2026	Plugins en mode recherche ; actions, mesures et attentes demandées par les LLM.
1.11.5	septembre 2026	Directives tolérantes au balisage : un mot-clé décoré, isolé sur sa ligne ou suivi d'un commentaire reste reconnu.

Version	Date	Apport
1.11.11	septembre 2026	Diagnostic de troncature mesuré sur les compteurs d'Ollama. Plugin chronogramme.py : un plugin peut conduire la séance.

11.4 Aide-mémoire du contrat

```
# ---- Metadonnees (optionnelles) ----
PLUGIN_NAME, PLUGIN_VERSION, PLUGIN_ENABLED

# ---- Hooks ----
list_variables() -> [(nom, description), ...]
collect(ctx)      -> {NOM: valeur}           # avant chaque tour
on_message(msg, ctx) -> {NOM: valeur} | None  # apres chaque tour
                    -> ({NOM: valeur}, ["avertissement"]) # forme longue

# ---- Actions (v1.11) ----
list_actions()    -> [(action_id, libelle, description), ...]
execute_action(action_id, params, ctx) -> {NOM: valeur} # budget 10 s

# ---- Directives ecrites par les LLM (seules sur leur ligne, MAJUSCULES) ----
ACTION: <nom>(param=valeur)
MESURE: <nom>(param=valeur)
ATTENDRE: <VARIABLE> <op> <valeur> delai=<secondes>
          op : > >= < <= = != contient
3 directives max/message | modes : auto | confirmer | off

# ---- ctx ----
tour, theme, speaker, role, sujet, mode_jdr, labels,
vars / session_vars, historique_court

# ---- msg ----
tour, speaker, role, texte, theme, ts, humain

# ---- Limites ----
timeout 2 s | 2 timeouts = desactivation | 160 car./valeur
12 variables | bloc 800 car. | markdown retire | valeurs vides ignorees

# ---- Fichier ----
plugins_dia\mon_plugin.py      actif
plugins_dia\mon_plugin.py.disabled  neutralise
plugins_dia\brouillon.py        ignore (prefixe souligne)
```

11.5 Corrigés des exercices

Exercice 1 — Entrée ou sortie ?

- a) `collect()` — c'est une donnée qui entre. Variable : `AUTONOMIE_PCO`, exprimée en heures et qualifiée (« 3 h, groupe electrogene EHPAD »).
- b) `on_message()` seul — rien n'est injecté, on compte et on écrit dans un fichier. Si l'on veut que le COD réagisse, on retourne en plus une variable du type `DEFAULT_RELEVE`.
- c) `collect()` — mais attention : le plugin ne ferme pas la fenêtre, il pose `NVIS_ETAT`. C'est l'injecteur, en lisant cette variable, qui en tire un événement. Un plugin décrit la situation, il ne pilote pas la narration.
- d) `on_message()` pour la collecte au fil de l'eau, et un traitement hors d-IA pour la transmission : un plugin n'ouvre pas une liaison Winlink pendant un tour de dialogue — le timeout de 2 secondes l'interdit. On écrit le SITREP retenu dans un fichier, et TCQ le transmet.

Exercice 2 — Diagnostiquer un plugin défaillant

Premier message : le fichier a été importé sans erreur, mais il n'expose ni `collect()` ni `on_message()`. Le plugin fait peut-être quelque chose d'utile, mais d-IA n'a aucun moyen de l'appeler. Correction : ajouter au moins l'un des deux hooks, avec la bonne signature — `collect(ctx)` prend un argument, `on_message(msg, ctx)` en prend deux.

Second message : le plugin a dépassé deux fois la limite de 2 secondes ; il est neutralisé pour le reste de la séance. La cause est presque toujours un accès lent (requête réseau, port série, gros fichier). Correction : déporter l'acquisition dans un programme tiers qui écrit un fichier, et faire lire ce fichier par le plugin. Le plugin ne doit jamais attendre une donnée : il prend ce qui est déjà là.

Exercice 3 — Audit de sécurité

Sécurité (ligne 3) : `subprocess` n'a rien à faire dans un plugin de lecture de données. Un plugin légitime de ce type utilise `json`, `re`, `datetime` et `pathlib`. La présence de `subprocess` sans justification écrite dans l'en-tête est un motif de refus.

Doctrine (ligne 6) : le plugin dépend d'un service web. Un exercice de black-out se joue hors ligne ; le jour où l'on en aura le plus besoin, la requête échouera. La donnée doit venir d'un fichier local, alimenté quand le réseau existe.

Technique (ligne 6 également) : `timeout=10` dépasse largement la limite de 2 secondes de d-IA. Le plugin sera neutralisé au deuxième tour, silencieusement du point de vue du stagiaire.

Réécriture : un programme tiers (tâche planifiée, GEOMAG-Observer) interroge l'API quand il peut et écrit `kp.json` ; le plugin se contente de :

```
import json
from pathlib import Path

FICHIER = Path(__file__).with_name("kp.json")

def collect(ctx):
    if not FICHIER.exists():
        return {}
    try:
        return {"KP_LOCAL": json.loads(FICHIER.read_text())["kp"]}
    except Exception:
        return {}
```

Exercice 4 — Piloter une séance à la main

- a) Deux variables suffisent, qualifiées pour que les modèles sachent quoi en faire :

```
SOC_PCO = 8 % (power station en fin d'autonomie)
```

```
LIAISON_Q0100 = perdue depuis 11h40 (pointage a refaire)
```

b) On les saisit au moment où la séquence 5 commence — le changement de séquence est annoncé dans le fil par une ligne de méta. La prise en compte est immédiate, y compris dialogue en cours. Si un plugin produit déjà SOC_PCO, la valeur manuelle l'emporte : c'est la règle, et c'est ce qui permet au formateur d'imposer une situation sans débrancher ses plugins.

c) Parce que modifier le sujet en pleine séance est impossible sans arrêter le dialogue, et parce que le sujet est le fil conducteur permanent : y écrire une situation transitoire la rendrait vraie pour tout l'exercice, y compris avant qu'elle ne survienne. Une variable de situation est datée par nature ; le sujet ne l'est pas.

Exercice 5 — Lire et valider une variable

```
# -*- coding: utf-8 -*-
"""energie_vars.py - autonomie energetique du PCO."""

import json
from pathlib import Path

PLUGIN_NAME = "Energie PCO"
PLUGIN_VERSION = "1.0"

FICHIER = Path(__file__).with_name("energie.json")
CAPACITE_WH = 1200.0

def list_variables():
    return [("AUTONOMIE_PCO", "Autonomie restante estimee (heures)"),
            ("ALERTE_ENERGIE", "Alerte si autonomie inferieure a 2 h")]

def _nombre(valeur):
    """Retourne un float, ou None si la valeur n'est pas exploitable."""
    try:
        return float(valeur)
    except (TypeError, ValueError):
        return None

def collect(ctx):
    if not FICHIER.exists():
        return {}, ["energie.json absent - aucune donnee energie injectee"]
    try:
        data = json.loads(FICHIER.read_text(encoding="utf-8"))
    except Exception:
        return {}, ["energie.json illisible - lecture ignoree"]

    soc = _nombre(data.get("soc_pct"))
    conso = _nombre(data.get("conso_w"))
    if soc is None or conso is None or conso <= 0:
        return {}, ["soc_pct ou conso_w absent ou non numerique"]

    heures = (CAPACITE_WH * max(0.0, min(100.0, soc)) / 100.0) / conso
    out = {"AUTONOMIE_PCO": f"{heures:.1f} h (SOC {soc:g} %, {conso:g} W)"}
    if heures < 2.0:
        out["ALERTE_ENERGIE"] = ("autonomie critique - prévoir relais "
                                "ou groupe electrogene")
    return out
```

Points notables : la validation passe par une fonction dédiée plutôt que par un try/except autour de tout ; le SOC est borné entre 0 et 100 avant calcul ; la valeur injectée rappelle les données sources, ce qui rend le trafic vérifiable ; et le fichier absent produit un avertissement, pas une erreur.

Exercice 6 — Workflow formateur complet

1. La veille, vérifier que le Kp est disponible : soit GEOMAG-Observer écrit propagation_relevés.json, soit vous le remplissez à la main depuis votre relevé du jour.

2. Ouvrir `plugins_dia\propagation_relevés.json` et renseigner `kp`, et si vous les avez `sfi`, `soc_pct`, `autonomie_h`.
3. Lancer d-IA. Menu Plugins E/S → Diagnostic plugins : vérifier les lignes [OK] pour `propagation_vars.py` et `sitrep_scoring.py`.
4. Cocher « Activer les plugins et variables de situation ».
5. Importer `formation_helios_noir_26_operateur.diapreset.json` (mode CHAT actif).
6. Vérifier les modèles du LLM1 et du LLM3 ; l'onglet IA 2 doit afficher « (humain) ».
7. Avant de démarrer : rouvrir le Diagnostic et lire le bloc des variables — c'est exactement ce que les modèles vont lire.
8. Remettre au stagiaire sa fiche de rôle et le rappel du format SITREP.
9. Démarrer. Observer sans intervenir ; le COD réclamera de lui-même les éléments manquants grâce à `DEFAULT_RELEVÉ`.
10. Arrêter, produire le SITREP PDF, ouvrir `logs\retex_sitrep_AAAAMMJ.csv`, trier par score et conduire le débriefing.

Le piège classique : oublier de cocher la case d'activation. Les plugins sont chargés, le diagnostic est vert, et pourtant rien n'est injecté. La ligne de méta « [Plugins E/S actifs : ...] » au lancement du dialogue est la confirmation à chercher.

Exercice 7 — Compléter la notation des SITREP

```
RE_MOYEN = re.compile(
    r"\bvara\b|\bnvis\b|\bqo-?100\b|\bpacket\b|\bwink\b|"
    r"\b1[34]\d[.,]\d{3}\b|\b\d{1,2}[.,]\d{3}s?(mhz|khz)\b",
    re.IGNORECASE)

# Dans _noter(), la brevete passe de 4 a 2 points :
if len(lignes) <= 6 and len(texte or "") <= 800:
    score += 2
else:
    defaults.append("message trop long")
if RE_MOYEN.search(texte or ""):
    score += 2
else:
    defaults.append("moyen de liaison non annonce")
```

L'en-tête du CSV ne change pas — la colonne `score_sur_20` reste sur 20, seule la répartition évolue. Et c'est précisément le problème si le fichier du jour contient déjà des lignes : deux barèmes différents y cohabiteraient sans que rien ne le signale. Deux solutions acceptables : versionner le nom du fichier (`retex_sitrep_AAAAMMJ_v2.csv`), ou ajouter une colonne bareme. La mauvaise solution est de ne rien faire en se disant qu'on s'en souviendra au débriefing.

Exercice 8 — Plugin « main courante TCQ »

```
# -*- coding: utf-8 -*-
"""tcq_maincourante.py - dernier trafic TCQ injecte dans la simulation."""

import json
import time
from pathlib import Path

PLUGIN_NAME = "Main courante TCQ"
PLUGIN_VERSION = "1.0"

FICHER = Path(__file__).with_name("maincourante.json")
SEUIL_SILENCE_S = 30 * 60

_ETAT = {"vus": 0}

def list_variables():
    return [
        ("DERNIER_MSG_RECU", "Expediteur et debut du dernier message TCQ"),
        ("MSG_EN_ATTENTE", "Messages recus non encore traites"),
```

```

        ("LIAISON_ACTIVE", "Moyen du dernier message reçu")]

def _charger():
    """Retourne la liste des messages, ou None si indisponible."""
    if not FICHIER.exists():
        return None
    try:
        data = json.loads(FICHIER.read_text(encoding="utf-8"))
    except Exception:
        return None
    return data if isinstance(data, list) else None

def collect(ctx):
    msgs = _charger()
    if not msgs:
        return {}
    avertis = []
    age = time.time() - FICHIER.stat().st_mtime
    if age > SEUIL_SILENCE_S:
        avertis.append(f"main courante TCQ figee depuis {int(age // 60)} min")

    dernier = msgs[-1]
    texte = str(dernier.get("texte", ""))[:100]
    out = {
        "DERNIER_MSG_RECU": f"{dernier.get('de', '?')} : {texte}",
        "MSG_EN_ATTENTE": max(0, len(msgs) - _ETAT["vus"]),
        "LIAISON_ACTIVE": dernier.get("moyen", "non precise"),
    }
    return out, avertis

def on_message(msg, ctx):
    """Quand l'operateur parle, on considere le trafic reçu comme traite."""
    if msg.get("speaker") == "LLM2":
        msgs = _charger() or []
        _ETAT["vus"] = len(msgs)
    return None

if __name__ == "__main__":
    FICHIER.write_text(json.dumps([
        {"ts": "2026-09-13T14:02:00", "de": "F1ABC", "vers": "F1XYZ",
         "moyen": "VARA FM", "texte": "EXERCICE - demande point de situation"}
    ], ensure_ascii=False), encoding="utf-8")
    print(collect({"tour": 1, "speaker": "LLM1", "vars": {}}))
    print(on_message({"speaker": "LLM2", "texte": "recu"}, {}))
    print(collect({"tour": 2, "speaker": "LLM1", "vars": {}}))

```

Trois points valent d'être relevés dans ce corrigé. Le compteur de messages en attente est remis à jour dans `on_message()` et non dans `collect()` : c'est la prise de parole de l'opérateur qui vaut traitement du trafic, pas la lecture d'une variable. L'avertissement de silence utilise la date de modification du fichier, pas son contenu : un fichier figé est une panne, et une panne doit se voir. Enfin, l'autotest écrit lui-même son jeu de données, ce qui rend le plugin testable sur un poste où TCQ n'est pas installé.

Exercice 9 — Une action et son attente

a) Deux actions, et surtout deux retours différents :

```

def list_actions():
    return [
        ("enregistrer_vent", "Lancer un enregistrement de 10 min",
         "Demarre l'enregistreur et rend la main immediatement"),
        ("mesure_vent", "Relever la vitesse du vent instantanee", "")]

def execute_action(action_id, params, ctx):
    if action_id == "enregistrer_vent":
        duree = int(params.get("duree", 600))
        _ETAT["fin"] = time.time() + duree          # on note l'echecance
        _lancer_enregistreur(duree)                 # Popen : non bloquant
        return {"ENREGISTREMENT": "en cours",
                "ACTION_RESULTAT": f"enregistrement de {duree} s lance"}
    if action_id == "mesure_vent":
        return {"VENT_KMH": _lire_anemometre()}
    return {}

```

```
def collect(ctx):
    fin = _ETAT.get("fin")
    if fin and time.time() >= fin:
        return {"ENREGISTREMENT": "termine", "VENT_KMH": _lire_anemometre()}
    return {"ENREGISTREMENT": "en cours" if fin else "inactif"}
```

b) La variable ENREGISTREMENT, mise à jour par collect() et non par l'action. Le LLM écrit :

```
ACTION: enregistrer_vent(duree=600)
ATTENDRE: ENREGISTREMENT = termine delai=700
```

Le délai est légèrement supérieur à la durée demandée : il faut laisser à la scrutation le temps de constater la fin. Un délai plus court n'est pas une erreur — il produit une issue « NON satisfait », que le modèle lira et pourra commenter.

c) Parce qu'une action dispose de 10 secondes. Une execute_action() qui attendrait 600 secondes serait coupée au dixième de son travail, le résultat serait ignoré, et au deuxième essai le plugin serait désactivé pour la séance. La règle est constante depuis le chapitre 3 : on déclenche, on rend la main, on constate plus tard. L'attente appartient au moteur, jamais au plugin.

d) L'attente expire au bout du délai et injecte « ENREGISTREMENT = termine NON satisfait : délai de 700 s dépassé (dernier relevé : ENREGISTREMENT = en cours) ». Le dialogue reprend, le modèle sait qu'il n'a pas sa mesure, et le fil en garde la trace. Rien ne reste bloqué — c'est précisément ce que garantit le plafond plugins_attente_max_s.

Exercice 10 — Audit d'une directive et d'un mode

a) Trois conditions étaient réunies : un plugin de domotique déclarait l'action groupe_electrogene_on, le mode d'exécution était « auto », et le preset d'exercice ne l'avait pas imposé à « off ». Le COD simulé a formulé une consigne de jeu parfaitement normale pour le scénario — et d-IA l'a exécutée sur du matériel réel.

C'est un incident de sûreté, pas un défaut d'écriture du persona : un scénario de crise EST fait de consignes d'action. Tant qu'un exercice fictif partage le même dispositif qu'un pilotage réel, c'est le dispositif qui doit trancher, pas la formulation.

b) Deux corrections, à deux niveaux :

- Dans le preset d'exercice : ajouter "plugins_actions_mode": "off" aux réglages. Le preset impose alors le mode à l'import, et l'exercice est protégé partout où il est rejoué — y compris sur le poste d'un autre opérateur.
- Dans d-IA : retirer le plugin de domotique du dossier plugins_dia\ pendant les séances d'exercice, ou le neutraliser en le renommant en .disabled. Un plugin absent ne peut pas être appelé par erreur.

La première correction voyage avec l'exercice, la seconde protège le poste. Les deux ensemble valent mieux que l'une des deux : un preset peut être modifié, un dossier peut être recopié.

c) Parce que renommer l'action ne change pas ce qu'elle fait. Le modèle demanderait groupe_electrogene_simule_on, d-IA appellerait le même execute_action(), et le même relais se fermerait. Le nom est une étiquette ; la sûreté tient au mode d'exécution et à la présence du plugin. C'est exactement le raisonnement de l'exercice 3 : ce qui compte n'est pas comment l'appel est nommé, mais ce qu'il déclenche.

Exercice 11 — Le relevé d'à côté

a) Celui du 7.8 : accepter le paramètre lieu= sur l'action de déclenchement mais l'ignorer sur l'action de lecture. Le fichier ne contenant qu'un relevé à la fois, mesure_humidite(lieu=combles) rendrait tranquillement la dernière valeur écrite, quelle qu'en soit l'origine. Le piège est d'autant plus vicieux ici que les trois capteurs sont dans le même bâtiment : les valeurs seront proches, donc l'erreur sera invisible.

b) Deux au minimum, distinctes : CAPTEUR_DEMANDE (ce que l'on a demandé) et CAPTEUR_RELEVÉ (ce que l'on a lu), plus RELEVÉ_AGE_MIN. Tant qu'elles sont séparées, l'écart se voit dans le fil. Une seule variable « capteur » les confondrait — et c'est exactement le défaut que l'on cherche à éviter.

c) Vous ne rendez AUCUNE valeur d'humidité. Vous déclenchez le relevé du capteur demandé, vous publiez CAPTEUR_DEMANDE et l'état de l'acquisition, et vous rendez une directive d'attente. Le fait que les deux autres capteurs soient frais n'a aucune importance : ils mesurent autre chose.

Une variante acceptable existe, celle du plugin v1.6 : si le relevé est bien celui du capteur demandé mais qu'il est ancien, rendez-le AVEC son âge et rafraîchissez en arrière-plan. La valeur est alors vraie, seulement datée — et l'opérateur dispose de tout ce qu'il faut pour en juger.

d) Que « faute de mieux » n'existe pas en matière de mesure. Rendre la valeur du capteur voisin, c'est produire une donnée fausse et la présenter comme une mesure ; l'absence de valeur, elle, est une information exacte que le modèle sait traiter — il le prouve au 7.8, où il refuse de répondre plutôt que d'inventer. Si la proximité des capteurs rend leurs valeurs interchangeables, alors c'est le découpage en trois capteurs qui est à revoir, pas la règle de fidélité.

Exercice 12 — Trois chiffres et un affichage

a) MESUREAGEMIN, qui devrait se lire MESURE_AGE_MIN. Ce n'est pas votre plugin qui se trompe : c'est le rendu Markdown antérieur à d-IA v1.11.2 qui prenait les soulignés intérieurs pour des marques d'italique et les effaçait. La directive s'exécutera donc parfaitement — le moteur, lui, lit le texte brut. Le danger n'est pas pour la machine mais pour le stagiaire qui recopie la ligne projetée et cherchera ensuite pourquoi son attente n'aboutit pas.

b) Parce que l'écart n'est pas le sujet. En publiant 62 vous retirez une information que la source fournissait, et un modèle de langage comble ce vide de façon crédible : c'est ainsi qu'un VENT_KMH = 20 est ressorti en « 20,1 km/h » au 7.9. Publiez 61.8. À l'inverse, si le capteur ne donne que des entiers, n'écrivez pas 62.0 : vous suggéreriez une précision inexistante.

c) Le lieu et l'âge. « HUMIDITE = 62 » ne dit ni de quel capteur elle vient, ni de quand elle date. MESURE_RESULTAT mentionne bien la cave, mais dans une phrase en prose — un modèle peut l'attribuer à la mauvaise valeur, et un lecteur pressé aussi. Les qualificatifs d'une mesure doivent être des variables à part entière : CAPTEUR_RELEVÉ et RELEVÉ_AGE_MIN, publiées à côté de la valeur, pas racontées autour d'elle.

Exercice 13 — La colonne qui ne doit pas sortir

a) Non. La règle « pas de divulgation » du § 3.1.1 du guide ne porte pas sur les seuls événements à venir : elle porte sur tout élément du montage. Or un objectif visé dit, en creux, ce que les joueurs sont censés faire — c'est une réaction attendue exprimée autrement. Un modèle qui lit « objectif : tester la bascule de vecteur sur perte de liaison » bascule de vecteur, et l'exercice n'aura rien mesuré.

b) Que ce qu'on lit au DEBEX et ce qu'on porte au chronogramme ne sont pas la même chose. Au DEBEX, on énonce l'objectif général — « entraîner l'opérateur à tenir un PCO isolé ». Au chronogramme, l'objectif est rattaché à une ligne précise, à un instant précis : le publier revient à dire au joueur ce qu'on attend de lui, maintenant. La différence de grain fait toute la différence.

c) Un cas de test qui déroule la séance et vérifie qu'aucune sortie — variable ou avertissement — ne contient le contenu de la colonne. C'est exactement ce que fait le cas n° 6 de test_chronogramme.py : le CSV de test porte des objectifs écrits « NE DOIT PAS APPARAÎTRE », et le test échoue si cette chaîne ressort où que ce soit. Une vérification à la relecture ne survit pas à la troisième modification du plugin.

d) Non, et la distinction vaut d'être comprise. Dans un exercice cadre, les joueurs SAVENT qu'ils jouent : tous les messages sont encadrés du mot EXERCICE, et le guide fait même figurer le schéma de la salle d'animation dans les documents remis aux joueurs, « pour qu'ils sachent qui appeler ». Ce qui doit rester caché n'est pas l'existence de l'animation, c'est ce qu'elle attend d'eux. INJECT_DE relève du premier, la réaction attendue du second.

Exercice 14 — Un pupitre mal placé

a) Le preset d'exercice impose "plugins_actions_mode": "off", comme il le doit. Dans ce mode, les directives sont journalisées mais rien n'est exécuté, et le catalogue d'actions n'est même pas proposé aux modèles. Le pupitre du formateur, construit sur des actions, disparaît avec elles.

b) Il retrouverait son pupitre, et il ouvrirait en même temps le catalogue aux trois modèles. Son plugin déclare terminer_exercice : un COD simulé, dans un scénario qui tourne mal, a toutes les raisons narratives d'écrire une directive pour y mettre fin. C'est le même incident que celui de l'exercice 10, transposé à la conduite de séance — et il ne dépend d'aucune formulation malheureuse du persona : un scénario de crise EST fait de consignes d'action.

c) Faire passer le pupitre par des VARIABLES DE SESSION. Elles sont saisies par l'opérateur, priment sur celles des plugins, sont prises en compte immédiatement, et ne figurent dans aucun catalogue : les modèles ne peuvent ni les lire comme des commandes disponibles, ni les écrire. Le critère est simple et se retient : une capacité qui appartient au formateur passe par une variable de session ; une capacité qui appartient aux modèles passe par une action.

d) Non. Une action en lecture seule ne change rien à l'état de la séance : au pire, un modèle demande où en est l'exercice et reçoit l'heure de jeu et l'avancement — des informations qu'il a déjà sous les yeux dans les DONNÉES DE SITUATION. Le critère n'est pas « action ou pas », mais « qui décide ». C'est le choix fait par chronogramme.py, qui ne déclare que chronogramme_etat.

— FIN DU DOCUMENT —